



Web Services Description Language (WSDL) Version 2.0 Part 1: Core Language

W3C Recommendation 26 June 2007

This version:

<http://www.w3.org/TR/2007/REC-wsdl20-20070626>

Latest version:

<http://www.w3.org/TR/wsdl20>

Previous version:

<http://www.w3.org/TR/2007/PR-wsdl20-20070523>

Editors:

Roberto Chinnici, Sun Microsystems
Jean-Jacques Moreau, Canon
Arthur Ryman, IBM
Sanjiva Weerawarana, WSO2

Please refer to the [errata](#) for this document, which may include some normative corrections.

This document is also available in these non-normative formats: [XHTML with Z Notation](#), [PDF](#), [PostScript](#), [XML](#), and [plain text](#).

See also [translations](#).

Copyright © 2007 W3C® ([MIT](#), [ERCIM](#), [Keio](#)), All Rights Reserved. W3C [liability](#), [trademark](#) and [document use](#) rules apply.

Abstract

This document describes the Web Services Description Language Version 2.0 (WSDL 2.0), an XML language for describing Web services. This specification defines the core language which can be used to describe Web services based on an abstract model of what the service offers. It also defines the conformance criteria for documents in this language.

Status of this Document

This section describes the status of this document at the time of its publication. Other documents may supersede this document. A list of current W3C publications and the latest revision of this technical report can be found in the [W3C technical reports index](#) at <http://www.w3.org/TR/>.

This is the [W3C Recommendation](#) of Web Services Description Language (WSDL) Version 2.0 Part 1: Core Language for review by W3C Members and other interested parties. It has been produced by the [Web Services Description Working Group](#), which is part of the [W3C Web Services Activity](#).

Please send comments about this document to the public public-ws-desc-comments@w3.org mailing list ([public archive](#)).

The Working Group released a test suite along with an [implementation report](#). A [diff-marked version against the previous version of this document](#) is available.

This document has been reviewed by W3C Members, by software developers, and by other W3C groups and interested parties, and is endorsed by the Director as a W3C Recommendation. It is a stable document and may be used as reference material or cited from another document. W3C's role in making the Recommendation is to draw attention to the specification and to promote its widespread deployment. This enhances the functionality and interoperability of the Web.

This document is governed by the [24 January 2002 CPP](#) as amended by the [W3C Patent Policy Transition Procedure](#). W3C maintains a [public list of any patent disclosures](#) made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent which the individual believes contains [Essential Claim\(s\)](#) must disclose the information in accordance with [section 6 of the W3C Patent Policy](#).

Table of Contents

1. [Introduction](#)
 - 1.1 [Service Description](#)
 - 1.2 [The Meaning of a Service Description](#)
 - 1.3 [Document Conformance](#)

- 1.4 [Notational Conventions](#)
 - 1.4.1 [RFC 2119 Keywords](#)
 - 1.4.2 [RFC 3986 Namespaces](#)
 - 1.4.3 [XML Schema anyURI](#)
 - 1.4.4 [Prefixes and Namespaces Used in This Specification](#)
 - 1.4.5 [Terms Used in This Specification](#)
 - 1.4.6 [XML Information Set Properties](#)
 - 1.4.7 [WSDL 2.0 Component Model Properties](#)
 - 1.4.8 [Z Notation](#)
 - 1.4.9 [BNF Pseudo-Schemas](#)
 - 1.4.10 [Assertions](#)
- 2. [Component Model](#)
 - 2.1 [Description](#)
 - 2.1.1 [The Description Component](#)
 - 2.1.2 [XML Representation of Description Component](#)
 - 2.1.2.1 [targetNamespace attribute information item](#)
 - 2.1.3 [Mapping Description's XML Representation to Component Properties](#)
 - 2.2 [Interface](#)
 - 2.2.1 [The Interface Component](#)
 - 2.2.2 [XML Representation of Interface Component](#)
 - 2.2.2.1 [name attribute information item with interface \[owner element\]](#)
 - 2.2.2.2 [extends attribute information item](#)
 - 2.2.2.3 [styleDefault attribute information item](#)
 - 2.2.3 [Mapping Interface's XML Representation to Component Properties](#)
 - 2.3 [Interface Fault](#)
 - 2.3.1 [The Interface Fault Component](#)
 - 2.3.2 [XML Representation of Interface Fault Component](#)
 - 2.3.2.1 [name attribute information item with fault \[owner element\]](#)
 - 2.3.2.2 [element attribute information item with fault \[owner element\]](#)
 - 2.3.3 [Mapping Interface Fault's XML Representation to Component Properties](#)
 - 2.4 [Interface Operation](#)
 - 2.4.1 [The Interface Operation Component](#)
 - 2.4.1.1 [Message Exchange Pattern](#)
 - 2.4.1.2 [Operation Style](#)
 - 2.4.2 [XML Representation of Interface Operation Component](#)
 - 2.4.2.1 [name attribute information item with operation \[owner element\]](#)
 - 2.4.2.2 [pattern attribute information item with operation \[owner element\]](#)
 - 2.4.2.3 [style attribute information item with operation \[owner element\]](#)
 - 2.4.3 [Mapping Interface Operation's XML Representation to Component Properties](#)
 - 2.5 [Interface Message Reference](#)
 - 2.5.1 [The Interface Message Reference Component](#)
 - 2.5.2 [XML Representation of Interface Message Reference Component](#)
 - 2.5.2.1 [messageLabel attribute information item with input or output \[owner element\]](#)
 - 2.5.2.2 [element attribute information item with input or output \[owner element\]](#)
 - 2.5.3 [Mapping Interface Message Reference's XML Representation to Component Properties](#)
 - 2.6 [Interface Fault Reference](#)
 - 2.6.1 [The Interface Fault Reference Component](#)
 - 2.6.2 [XML Representation of Interface Fault Reference](#)
 - 2.6.2.1 [ref attribute information item with infault, or outfault \[owner element\]](#)
 - 2.6.2.2 [messageLabel attribute information item with infault, or outfault \[owner element\]](#)
 - 2.6.3 [Mapping Interface Fault Reference's XML Representation to Component Properties](#)
 - 2.7 [Binding](#)
 - 2.7.1 [The Binding Component](#)
 - 2.7.2 [XML Representation of Binding Component](#)
 - 2.7.2.1 [name attribute information item with binding \[owner element\]](#)
 - 2.7.2.2 [interface attribute information item with binding \[owner element\]](#)
 - 2.7.2.3 [type attribute information item with binding \[owner element\]](#)
 - 2.7.2.4 [Binding extension elements](#)
 - 2.7.3 [Mapping Binding's XML Representation to Component Properties](#)
 - 2.8 [Binding Fault](#)
 - 2.8.1 [The Binding Fault Component](#)
 - 2.8.2 [XML Representation of Binding Fault Component](#)
 - 2.8.2.1 [ref attribute information item with fault \[owner element\]](#)
 - 2.8.2.2 [Binding Fault extension elements](#)
 - 2.8.3 [Mapping Binding Fault's XML Representation to Component Properties](#)
 - 2.9 [Binding Operation](#)
 - 2.9.1 [The Binding Operation Component](#)
 - 2.9.2 [XML Representation of Binding Operation Component](#)
 - 2.9.2.1 [ref attribute information item with operation \[owner element\]](#)
 - 2.9.2.2 [Binding Operation extension elements](#)
 - 2.9.3 [Mapping Binding Operation's XML Representation to Component Properties](#)
 - 2.10 [Binding Message Reference](#)
 - 2.10.1 [The Binding Message Reference Component](#)

- 2.10.2 [XML Representation of Binding Message Reference Component](#)
 - 2.10.2.1 [messageLabel attribute information item with input or output \[owner element\]](#)
 - 2.10.2.2 [Binding Message Reference extension elements](#)
- 2.10.3 [Mapping Binding Message Reference's XML Representation to Component Properties](#)
- 2.11 [Binding Fault Reference](#)
 - 2.11.1 [The Binding Fault Reference Component](#)
 - 2.11.2 [XML Representation of Binding Fault Reference Component](#)
 - 2.11.2.1 [ref attribute information item with infault or outfault \[owner element\]](#)
 - 2.11.2.2 [messageLabel attribute information item with infault or outfault \[owner element\]](#)
 - 2.11.2.3 [Binding Fault Reference extension elements](#)
 - 2.11.3 [Mapping Binding Fault Reference's XML Representation to Component Properties](#)
- 2.12 [Service](#)
 - 2.12.1 [The Service Component](#)
 - 2.12.2 [XML Representation of Service Component](#)
 - 2.12.2.1 [name attribute information item with service \[owner element\]](#)
 - 2.12.2.2 [interface attribute information item with service \[owner element\]](#)
 - 2.12.3 [Mapping Service's XML Representation to Component Properties](#)
- 2.13 [Endpoint](#)
 - 2.13.1 [The Endpoint Component](#)
 - 2.13.2 [XML Representation of Endpoint Component](#)
 - 2.13.2.1 [name attribute information item with endpoint \[owner element\]](#)
 - 2.13.2.2 [binding attribute information item with endpoint \[owner element\]](#)
 - 2.13.2.3 [address attribute information item with endpoint \[owner element\]](#)
 - 2.13.2.4 [Endpoint extension elements](#)
 - 2.13.3 [Mapping Endpoint's XML Representation to Component Properties](#)
- 2.14 [XML Schema 1.0 Simple Types Used in the Component Model](#)
- 2.15 [Equivalence of Components](#)
- 2.16 [Symbol Spaces](#)
- 2.17 [QName resolution](#)
- 2.18 [Comparing URIs and IRIs](#)
- 3. [Types](#)
 - 3.1 [Using W3C XML Schema Definition Language](#)
 - 3.1.1 [Importing XML Schema](#)
 - 3.1.1.1 [namespace attribute information item](#)
 - 3.1.1.2 [schemaLocation attribute information item](#)
 - 3.1.2 [Inlining XML Schema](#)
 - 3.1.3 [References to Element Declarations and Type Definitions](#)
 - 3.2 [Using Other Schema Languages](#)
 - 3.3 [Describing Messages that Refer to Services and Endpoints](#)
 - 3.3.1 [wsdlx:interface attribute information item](#)
 - 3.3.2 [wsdlx:binding attribute information item](#)
 - 3.3.3 [wsdlx:interface and wsdlx:binding Consistency](#)
 - 3.3.4 [Use of wsdlx:interface and wsdlx:binding with xs:anyURI](#)
- 4. [Modularizing WSDL 2.0 descriptions](#)
 - 4.1 [Including Descriptions](#)
 - 4.1.1 [location attribute information item with include \[owner element\]](#)
 - 4.2 [Importing Descriptions](#)
 - 4.2.1 [namespace attribute information item](#)
 - 4.2.2 [location attribute information item with import \[owner element\]](#)
 - 4.3 [Extensions](#)
- 5. [Documentation](#)
- 6. [Language Extensibility](#)
 - 6.1 [Element-based Extensibility](#)
 - 6.1.1 [Mandatory extensions](#)
 - 6.1.2 [required attribute information item](#)
 - 6.2 [Attribute-based Extensibility](#)
 - 6.3 [Extensibility Semantics](#)
- 7. [Locating WSDL 2.0 Documents](#)
 - 7.1 [wsdl:wsdlLocation attribute information item](#)
- 8. [Conformance](#)
 - 8.1 [XML Information Set Conformance](#)
- 9. [XML Syntax Summary \(Non-Normative\)](#)
- 10. [References](#)
 - 10.1 [Normative References](#)
 - 10.2 [Informative References](#)

Appendices

- A. [The application/wsdl+xml Media Type](#)
 - A.1 [Registration](#)
 - A.2 [Fragment Identifiers](#)
 - A.2.1 [The Description Component](#)
 - A.2.2 [The Element Declaration Component](#)

- A.2.3 [The Type Definition Component](#)
 - A.2.4 [The Interface Component](#)
 - A.2.5 [The Interface Fault Component](#)
 - A.2.6 [The Interface Operation Component](#)
 - A.2.7 [The Interface Message Reference Component](#)
 - A.2.8 [The Interface Fault Reference Component](#)
 - A.2.9 [The Binding Component](#)
 - A.2.10 [The Binding Fault Component](#)
 - A.2.11 [The Binding Operation Component](#)
 - A.2.12 [The Binding Message Reference Component](#)
 - A.2.13 [The Binding Fault Reference Component](#)
 - A.2.14 [The Service Component](#)
 - A.2.15 [The Endpoint Component](#)
 - A.2.16 [Extension Components](#)
 - A.3 [Security considerations](#)
 - B. [Acknowledgements](#) (Non-Normative)
 - C. [IRI-References for WSDL 2.0 Components](#) (Non-Normative)
 - C.1 [WSDL 2.0 IRIs](#)
 - C.2 [Canonical Form for WSDL 2.0 Component Designators](#)
 - C.3 [Example](#)
 - D. [Component Summary](#) (Non-Normative)
 - E. [Assertion Summary](#) (Non-Normative)
-

1. Introduction

Web Services Description Language Version 2.0 (WSDL 2.0) provides a model and an XML format for describing Web services. WSDL 2.0 enables one to separate the description of the abstract functionality offered by a service from concrete details of a service description such as “how” and “where” that functionality is offered.

This specification defines a language for describing the abstract functionality of a service as well as a framework for describing the concrete details of a service description. It also defines the conformance criteria for documents in this language.

The companion specification, *Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts* [[WSDL 2.0 Adjuncts](#)] describes extensions for message exchange patterns, operation safety, operation styles and binding extensions (for SOAP [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)] and HTTP [[IETF RFC 2616](#)]).

1.1 Service Description

WSDL 2.0 describes a Web service in two fundamental stages: one abstract and one concrete. Within each stage, the description uses a number of constructs to promote reusability of the description and to separate independent design concerns.

At an abstract level, WSDL 2.0 describes a Web service in terms of the messages it sends and receives; messages are described independent of a specific wire format using a type system, typically XML Schema.

An *operation* associates a message exchange pattern with one or more messages. A *message exchange pattern* identifies the sequence and cardinality of messages sent and/or received as well as who they are logically sent to and/or received from. An *interface* groups together operations without any commitment to transport or wire format.

At a concrete level, a *binding* specifies transport and wire format details for one or more interfaces. An *endpoint* associates a network address with a binding. And finally, a *service* groups together endpoints that implement a common interface.

1.2 The Meaning of a Service Description

A WSDL 2.0 service description indicates how potential clients are intended to interact with the described service. It represents an assertion that the described service fully implements and conforms to what the WSDL 2.0 document describes. For example, as further explained in section [6.1.1 Mandatory extensions](#), if the WSDL 2.0 document specifies a particular optional extension, the functionality implied by that extension is only optional to the client. It must be supported by the Web service.

A WSDL 2.0 interface describes potential interactions with a Web service, not required interactions. The declaration of an operation in a WSDL 2.0 interface is not an assertion that the interaction described by the operation must occur. Rather it is an assertion that if such an interaction is (somehow) initiated, then the declared operation describes how that interaction is intended to occur.

1.3 Document Conformance

An *element information item* (as defined in [[XML Information Set](#)]) whose namespace name is “<http://www.w3.org/ns/wsdli>” and whose local part is *description* conforms to this specification if it is valid according to the XML Schema for that element as defined by this specification (<http://www.w3.org/2007/06/wsdli/wsdli20.xsd>) and additionally adheres to all the constraints contained in this specification and conforms to the specifications of any extensions contained in it. Such a conformant *element information item* constitutes a *WSDL 2.0 document*.

The definition of the WSDL 2.0 language is based on the XML Information Set [[XML Information Set](#)] but also imposes many semantic constraints over and above structural conformance to this XML Infoset. In order to precisely describe these constraints, and as an aid in precisely defining the meaning of each WSDL 2.0 document, the WSDL 2.0 specification defines a component model [2. Component Model](#) as an additional layer of abstraction above the XML Infoset. Constraints and meaning are defined in terms of this component model, and the definition of each component includes a mapping that specifies how values in the component model are derived from corresponding items in the XML Infoset.

An XML 1.0 document that is valid with respect to the WSDL 2.0 XML Schema and that maps to a valid WSDL 2.0 Component Model is conformant to the WSDL 2.0 specification.

1.4 Notational Conventions

All parts of this specification are normative, with the EXCEPTION of notes, pseudo-schemas, examples, and sections explicitly marked as “Non-Normative”.

1.4.1 RFC 2119 Keywords

The keywords “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119 [[IETF RFC 2119](#)].

1.4.2 RFC 3986 Namespaces

Namespace names of the general form:

- “http://example.org/...” and
- “http://example.com/...”

represent application or context-dependent URIs [[IETF RFC 3986](#)].

1.4.3 XML Schema anyURI

This specification uses the XML Schema type `xs:anyURI` (see [[XML Schema: Datatypes](#)]). It is defined so that `xs:anyURI` values are essentially IRIs (see [[IETF RFC 3987](#)]). The conversion from `xs:anyURI` values to an actual URI is via an escaping procedure defined by (see [[XLink 1.0](#)]), which is identical in most respects to IRI Section 3.1 (see [[IETF RFC 3987](#)]).

For interoperability, WSDL authors are advised to avoid the US-ASCII characters: “<”, “>”, “'”, space, “{”, “}”, “|”, “\”, “^”, and “~”, which are allowed by the `xs:anyURI` type, but disallowed in IRIs.

1.4.4 Prefixes and Namespaces Used in This Specification

This specification uses predefined namespace prefixes throughout; they are given in the following list. Note that the choice of any namespace prefix is arbitrary and not semantically significant (see [[XML Namespaces](#)]).

Table 1-1. Prefixes and Namespaces used in this specification

Prefix	Namespace	Notes
wsdl	“http://www.w3.org/ns/wsdl”	Defined by this specification.
wsdli	“http://www.w3.org/ns/wsdl-instance”	Defined by this specification 7.1 wsdli:wsdlLocation attribute information item .
wsdlx	“http://www.w3.org/ns/wsdl-extensions”	Defined by this specification 3.3 Describing Messages that Refer to Services and Endpoints .
wrpc	“http://www.w3.org/ns/wsdl/rpc”	Defined by WSDL 2.0: Adjuncts [WSDL 2.0 Adjuncts].
wsoap	“http://www.w3.org/ns/wsdl/soap”	Defined by WSDL 2.0: Adjuncts [WSDL 2.0 Adjuncts].
whhttp	“http://www.w3.org/ns/wsdl/http”	Defined by WSDL 2.0: Adjuncts [WSDL 2.0 Adjuncts].
xs	“http://www.w3.org/2001/XMLSchema”	Defined in the W3C XML Schema specification [XML Schema: Structures], [XML Schema: Datatypes].
xsi	“http://www.w3.org/2001/XMLSchema-instance”	Defined in the W3C XML Schema specification [XML Schema: Structures], [XML Schema: Datatypes].

1.4.5 Terms Used in This Specification

This section describes the terms and concepts introduced in Part 1 of the WSDL Version 2.0 specification (this document).

Actual Value

As in [[XML Schema: Structures](#)], the expression “actual value” is used to refer to the member of the value space of the simple type definition associated with an attribute information item which corresponds to its

normalized value. This will often be a string, but may also be an integer, a boolean, an IRI-reference, etc.

Inlined Schema

An XML schema that is defined in the `wsdl:types` *element information item* of a WSDL 2.0 description. For example, an XML Schema defined in an `xs:schema` *element information item* [3.1.2 Inlining XML Schema](#).

1.4.6 XML Information Set Properties

This specification refers to properties in the XML Information Set [[XML Information Set](#)]. Such properties are denoted by square brackets, e.g. [children], [attributes].

1.4.7 WSDL 2.0 Component Model Properties

This specification defines and refers to properties in the WSDL 2.0 Component Model [2. Component Model](#). Such properties are denoted by curly brackets, e.g. {name}, {interfaces}.

This specification uses a consistent naming convention for component model properties that refer to components. If a property refers to a required or optional component, then the property name is the same as the component name. If a property refers to a set of components, then the property name is the pluralized form of the component name.

1.4.8 Z Notation

Z Notation [[Z Notation Reference Manual](#)] was used in the development of this specification. Z Notation is a formal specification language that is based on standard mathematical notation. The Z Notation for this specification has been verified using the Fuzz 2000 type-checker [[Fuzz 2000](#)].

Since Z Notation is not widely known, it is not included in the normative version of this specification. However, it is included in a [non-normative version](#) which allows to dynamically hide and show the Z Notation. Browsers correctly display the mathematical Unicode characters, provided that the required fonts are installed. Mathematical fonts for Mozilla Firefox can be downloaded from the [Mozilla Web site](#).

The Z Notation was used to improve the quality of the normative text that defines the Component Model, and to help ensure that the test suite covered all important rules implied by the Component Model. However, the Z Notation is non-normative, so any conflict between it and the normative text is an error in the Z Notation. Readers and implementers may nevertheless find the Z Notation useful in cases where the normative text is unclear.

There are two elements of Z Notation syntax that conflict with the notational conventions described in the preceding sections. In Z Notation, square brackets are used to introduce basic sets, e.g. [ID], which conflicts with the use of square brackets to denote XML Information Set properties [1.4.6 XML Information Set Properties](#). Also, in Z Notation, curly brackets are used to denote set display and set comprehension, e.g. {1, 2, 3}, which conflicts with the use of curly brackets to denote WSDL 2.0 Component Model properties [1.4.7 WSDL 2.0 Component Model Properties](#). However, the intended meaning of square and curly brackets should be clear from their context and this minor notational conflict should not cause any confusion.

1.4.9 BNF Pseudo-Schemas

Pseudo-schemas are provided for each component, before the description of the component. They use BNF-style conventions for attributes and elements: "?" denotes optionality (i.e. zero or one occurrences), "*" denotes zero or more occurrences, "+" one or more occurrences, "[" and "]" are used to form groups, and "|" represents choice. Attributes are conventionally assigned a value which corresponds to their type, as defined in the normative schema. Elements with simple content are conventionally assigned a value which corresponds to the type of their content, as defined in the normative schema. Pseudo schemas do not include extension points for brevity.

```
<!-- sample pseudo-schema -->
<defined_element
  required_attribute_of_type_string="xs:string"
  optional_attribute_of_type_int="xs:int"? >
  <required_element />
  <optional_element />?
  <one_or_more_of_these_elements />+
  [ <choice_1 /> | <choice_2 /> ]*
</defined_element>
```

1.4.10 Assertions

Assertions about WSDL 2.0 documents and components that are not enforced by the normative XML schema for WSDL 2.0 are marked by a dagger symbol (†) at the end of a sentence. Each assertion has been assigned a unique identifier that consists of a descriptive textual prefix and a unique numeric suffix. The numeric suffixes are assigned sequentially and never reused so there may be gaps in the sequence. The assertion identifiers MAY be used by implementations of this specification for any purpose, e.g. error reporting.

The assertions and their identifiers are summarized in section [E. Assertion Summary](#).

2. Component Model

This section describes the conceptual model of WSDL 2.0 as a set of components with attached properties, which collectively describe a Web service. This model is called the *Component Model* of WSDL 2.0. A *valid WSDL 2.0*

component model is a set of WSDL 2.0 components and properties that satisfy all the requirements given in this specification as indicated by keywords whose interpretation is defined by RFC 2119 [[IETF RFC 2119](#)].

Components are typed collections of properties that correspond to different aspects of Web services. Each subsection herein describes a different type of component, its defined properties, and its representation as an XML Infoset [[XML Information Set](#)].

Properties are unordered and unique with respect to the component they are associated with. Individual properties' definitions may constrain their content (e.g., to a typed value, another component, or a set of typed values or components), and components may require the presence of a property to be considered conformant. Such properties are marked as REQUIRED, whereas those that are not required to be present are marked as OPTIONAL. By convention, when specifying the mapping rules from the XML Infoset representation of a component to the component itself, an optional property that is absent in the component in question is described as being "empty". Unless otherwise specified, when a property is identified as being a collection (a set or a list), its value may be a 0-element (empty) collection. In order to simplify the presentation of the rules that deal with sets of components, for all OPTIONAL properties whose type is a set, the absence of such a property from a component MUST be treated as semantically equivalent to the presence of a property with the same name and whose value is the empty set. In other words, every OPTIONAL set-valued property MUST be assumed to have the empty set as its default value, to be used in case the property is absent.

Component definitions are serializable in XML 1.0 format but are independent of any particular serialization of the component model. Component definitions use a subset (see [2.14 XML Schema 1.0 Simple Types Used in the Component Model](#)) of the simple types defined by the XML Schema 1.0 specification [[XML Schema: Datatypes](#)].

In addition to the direct XML Infoset representation described here, the component model allows components external to the Infoset through the mechanisms described in [4. Modularizing WSDL 2.0 descriptions](#).

A component model can be extracted from a given XML Infoset which conforms to the XML Schema for WSDL 2.0 by recursively mapping Information Items to their identified components, starting with the `wsdl:description` *element information item*. This includes the application of the mechanisms described in [4. Modularizing WSDL 2.0 descriptions](#).

This document does not specify a means of producing an XML Infoset representation from a component model instance. In particular, there are in general many valid ways to modularize a given component model instance into one or more XML Infosets.

2.1 Description

2.1.1 The Description Component

At a high level, the [Description](#) component is just a container for two categories of components: WSDL 2.0 components and type system components.

WSDL 2.0 components are interfaces, bindings and services. Type system components are element declarations and type definitions.

Type system components describe the constraints on a message's content. By default, these constraints are expressed in terms of the [[XML Information Set](#)], i.e. they define the [local name], [namespace name], [children] and [attributes] properties of an *element information item*. Type systems based upon other data models are generally accommodated by extensions to WSDL 2.0; see [6. Language Extensibility](#). In the case where they define information equivalent to that of a XML Schema global element declaration, they can be treated as if they were such a declaration.

This specification does not define the behavior of a WSDL 2.0 document that uses multiple schema languages for describing type system components simultaneously.

An Element Declaration component defines the name and content model of an *element information item* such as that defined by an XML Schema global element declaration. It has a {name} property that is the QName of the *element information item* and a {system} property that is the namespace IRI of the extension *element information items* for the type system, e.g. the namespace of XML Schema.

A Type Definition component defines the content model of an *element information item* such as that defined by an XML Schema global type definition. It has a {name} property that is the QName of the type and a {system} property that is the namespace IRI of the extension *element information items* for the type system, e.g. the namespace of XML Schema.

[Interface](#), [Binding](#), [Service](#), [Element Declaration](#), and [Type Definition](#) components are directly contained in the [Description](#) component and are referred to as *top-level components*. The top-level WSDL 2.0 components contain other components, e.g. [Interface Operation](#) and [Endpoint](#), which are referred to as *nested components*. Nested components may contain other nested components. The component that contains a nested component is referred to as the *parent* of the nested component. Nested components have a {parent} property that is a reference to their parent component.

The properties of the Description component are as follows:

- {interfaces} OPTIONAL. A set of [Interface](#) components.
- {bindings} OPTIONAL. A set of [Binding](#) components.
- {services} OPTIONAL. A set of [Service](#) components.
- {element declarations} OPTIONAL. A set of [Element Declaration](#) components.
- {type definitions} REQUIRED. A set of [Type Definition](#) components.

The set of top-level components contained in the [Description](#) component associated with an initial WSDL 2.0 document consists of the components defined in the initial document, plus the components associated with the WSDL 2.0 documents that the initial document includes, plus the components defined by other WSDL 2.0 documents in the namespaces that the initial document imports. The component model makes no distinction between the components that are defined in the initial document versus those that are defined in the included documents or imported namespaces. However, any WSDL 2.0 document that contains component definitions that refer by QName to WSDL 2.0 components that belong to a different namespace MUST contain a `wsdl:import` *element information item* for that namespace (see [4.2 Importing Descriptions](#)). Furthermore, all QName references, whether to the same or to different namespaces must resolve to components (see [2.17 QName resolution](#)).

When using the XML Schema language to describe type system components, the inclusion of [Element Declaration](#) components and [Type Definition](#) components in a [Description](#) component is governed by the rules in [3.1 Using W3C XML Schema Definition Language](#).

In addition to WSDL 2.0 components and type system components, additional extension components MAY be added via extensibility [6. Language Extensibility](#). Further, additional properties to WSDL 2.0 and type system components MAY also be added via extensibility.

2.1.2 XML Representation of Description Component

```
<description
  targetNamespace="xs:anyURI" >
  <documentation />*
  [ <import /> | <include /> ]*
  <types />?
  [ <interface /> | <binding /> | <service /> ]*
</description>
```

WSDL 2.0 descriptions are represented in XML by one or more WSDL 2.0 Information Sets (Infosets), that is one or more *description element information items*. A WSDL 2.0 Infoset contains representations for a collection of WSDL 2.0 components that share a common target namespace and zero or more `wsdl:import` *element information items* [4.2 Importing Descriptions](#) that correspond to a collection with components from multiple target namespaces.

The components directly defined or included within a [Description](#) component are said to belong to the same *target namespace*. The target namespace therefore groups a set of related component definitions and represents an unambiguous name for the intended semantics of the collection of components. The value of the `targetNamespace` *attribute information item* SHOULD be dereferencable.[‡] It SHOULD resolve to a human or machine processable document that directly or indirectly defines the intended semantics of those components.[‡] It MAY resolve to a WSDL 2.0 document that provides service description information for that namespace.[‡]

If a WSDL 2.0 document is split into multiple WSDL 2.0 documents (which may be combined as needed via [4.1 Including Descriptions](#)), then the `targetNamespace` *attribute information item* SHOULD resolve to a master WSDL 2.0 document that includes all the WSDL 2.0 documents needed for that service description.[‡] This approach enables the WSDL 2.0 component designator fragment identifiers to be properly resolved.

Components that belong to imported namespaces have different target namespace values than that of the importing WSDL 2.0 document. Thus importing is the mechanism to use components from one namespace in the definition of components from another namespace.

Note that each WSDL 2.0 document or type system component of the same kind must be uniquely identified by its qualified name. That is, if two distinct components of the same kind ([Interface](#), [Binding](#), etc.) are in the same target namespace, then their QNames MUST be unique. However, different kinds of components (e.g., an [Interface](#) component and a [Binding](#) component) MAY have the same QName. Thus, QNames of components must be unique within the space of those components in a given target namespace.

The *description element information item* has the following Infoset properties:

- A [local name] of `description`.
- A [namespace name] of `"http://www.w3.org/ns/wsdl"`.
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED `targetNamespace` *attribute information item* as described below in [2.1.2.1 targetNamespace attribute information item](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"`.
- Zero or more *element information items* amongst its [children], in order as follows:[‡]
 1. Zero or more `documentation` *element information items* (see [5. Documentation](#)).
 2. Zero or more *element information items* from among the following, in any order:
 - Zero or more `include` *element information items* (see [4.1 Including Descriptions](#))
 - Zero or more `import` *element information items* (see [4.2 Importing Descriptions](#))
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"`.
 3. An OPTIONAL `types` *element information item* (see [3. Types](#)).
 4. Zero or more *element information items* from among the following, in any order:
 - `interface` *element information items* (see [2.2.2 XML Representation of Interface Component](#)).

- binding *element information items* (see [2.7.2 XML Representation of Binding Component](#)).
- service *element information items* (see [2.12.2 XML Representation of Service Component](#)).
- Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdli".

2.1.2.1 targetNamespace attribute information item

The targetNamespace *attribute information item* defines the namespace affiliation of top-level components defined in this description *element information item*. [Interface](#), [Binding](#) and [Service](#) are top-level components.

The targetNamespace *attribute information item* has the following Infoset properties:

- A [local name] of targetNamespace
- A [namespace name] which has no value

The type of the targetNamespace *attribute information item* is *xs:anyURI*. Its value MUST be an absolute IRI (see [[IETF RFC 3987](#)]) and should be dereferencable.[‡]

2.1.3 Mapping Description's XML Representation to Component Properties

The mapping from the XML Representation of the description *element information item* (see [2.1.2 XML Representation of Description Component](#)) to the properties of the [Description](#) component is described in [Table 2-1](#).

Table 2-1. Mapping from XML Representation to Description Component Properties

Property	Value
{interfaces}	The set of Interface components corresponding to all the interface <i>element information items</i> in the [children] of the description <i>element information item</i> , if any, plus any included (via wsdl:include) or imported (via wsdl:import) Interface components (see 4. Modularizing WSDL 2.0 descriptions).
{bindings}	The set of Binding components corresponding to all the binding <i>element information items</i> in the [children] of the description <i>element information item</i> , if any, plus any included (via wsdl:include) or imported (via wsdl:import) Binding components (see 4. Modularizing WSDL 2.0 descriptions).
{services}	The set of Service components corresponding to all the service <i>element information items</i> in the [children] of the description <i>element information item</i> , if any, plus any included (via wsdl:include) or imported (via wsdl:import) Service components (see 4. Modularizing WSDL 2.0 descriptions).
{element declarations}	The set of Element Declaration components corresponding to all the element declarations defined as descendants of the types <i>element information item</i> , if any, plus any included (via xs:include) or imported (via xs:import) Element Declaration components. At a minimum this will include all the global element declarations defined by XML Schema <i>element information items</i> . It MAY also include any declarations from some other type system which describes the [local name], [namespace name], [attributes] and [children] properties of an <i>element information item</i> . Each XML Schema element declaration MUST have a unique QName. [‡]
{type definitions}	The set of Type Definition components corresponding to all the type definitions defined as descendants of the types <i>element information item</i> , if any, plus any included (via xs:include) or imported (via xs:import) Type Definition components. In addition, the built-in datatypes defined by XML Schema Part 2: Datatypes Second Edition [XML Schema: Datatypes], namely the nineteen primitive datatypes xs:string, xs:boolean, xs:decimal, xs:float, xs:double, xs:duration, xs:dateTime, xs:time, xs:date, xs:gYearMonth, xs:gYear, xs:gMonthDay, xs:gDay, xs:gMonth, xs:hexBinary, xs:base64Binary, xs:anyURI, xs:QName, xs:NOTATION, and the twenty-five derived datatypes xs:normalizedString, xs:token, xs:language, xs:NMTOKEN, xs:NMTOKENS, xs:Name, xs:NCName, xs:ID, xs:IDREF, xs:IDREFS, xs:ENTITY, xs:ENTITIES, xs:integer, xs:nonPositiveInteger, xs:negativeInteger, xs:long, xs:int, xs:short, xs:byte, xs:nonNegativeInteger, xs:unsignedLong, xs:unsignedInt, xs:unsignedShort, xs:unsignedByte, xs:positiveInteger. The set MAY also include any definitions from some other type system which describes the [attributes] and [children] properties of an <i>element information item</i> . Each XML Schema type definition MUST have a unique QName. [‡]

2.2 Interface

2.2.1 The Interface Component

An [Interface](#) component describes sequences of messages that a service sends and/or receives. It does this by grouping related messages into operations. An operation is a sequence of input and output messages, and an interface is a set of operations.

An interface can optionally extend one or more other interfaces. To avoid circular definitions, an interface **MUST NOT** appear in the set of interfaces it extends, either directly or indirectly. [‡] The set of operations available in an interface includes all the operations defined by the interfaces it extends directly or indirectly, together with any operations it directly defines. The operations directly defined on an interface are referred to as the *declared* operations of the interface. In the process, operation components that are equivalent per [2.15 Equivalence of Components](#) are treated as one single component. The interface extension mechanism behaves in a similar way for all other components that can be defined inside an interface, namely [Interface Fault](#) components.

Interfaces are named constructs and can be referred to by QName (see [2.17 QName resolution](#)). For instance, [Binding](#) components refer to interfaces in this way.

The properties of the Interface component are as follows:

- {name} REQUIRED. An *xs:QName*.
- {extended interfaces} OPTIONAL. A set of declared [Interface](#) components which this interface extends.
- {interface faults} OPTIONAL. The set of declared [Interface Fault](#) components. Note that the namespace name of the {name} property of each [Interface Fault](#) in this set is the same as the namespace name of the {name} property of this [Interface](#) component.
- {interface operations} OPTIONAL. A set of declared [Interface Operation](#) components. Note that the namespace name of the {name} property of each [Interface Operation](#) in this set is the same as the namespace name of the {name} property of this [Interface](#) component.

For each [Interface](#) component in the {interfaces} property of a [Description](#) component, the {name} property **MUST** be unique.[‡]

2.2.2 XML Representation of Interface Component

```
<description>
  <interface
    name="xs:NCName"
    extends="List of xs:QName"?
    styleDefault="List of xs:anyURI"? >
    <documentation />*
    [ <fault /> | <operation /> ]*
  </interface>
</description>
```

The XML representation for an [Interface](#) component is an *element information item* with the following Infoset properties:

- A [local name] of interface
- A [namespace name] of "http://www.w3.org/ns/wsdl"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *name attribute information item* as described below in [2.2.2.1 name attribute information item with interface \[owner element\]](#).
 - An OPTIONAL *extends attribute information item* as described below in [2.2.2.2 extends attribute information item](#).
 - An OPTIONAL *styleDefault attribute information item* as described below in [2.2.2.3 styleDefault attribute information item](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information items* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).
 2. Zero or more *element information items* from among the following, in any order:
 - Zero or more *fault element information items* [2.3.2 XML Representation of Interface Fault Component](#).
 - Zero or more *operation element information items* [2.4.2 XML Representation of Interface Operation Component](#).
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".

2.2.2.1 name attribute information item with interface [owner element]

The *name attribute information item* together with the *targetNamespace attribute information item* of the [parent] *description element information item* forms the QName of the interface.

The *name attribute information item* has the following Infoset properties:

- A [local name] of name
- A [namespace name] which has no value

The type of the name *attribute information item* is *xs:NCName*.

2.2.2.2 extends *attribute information item*

The extends *attribute information item* lists the interfaces that this interface derives from.

The extends *attribute information item* has the following Infoset properties:

- A [local name] of extends
- A [namespace name] which has no value

The type of the extends *attribute information item* is a whitespace-separated list of *xs:QName*.

The list of *xs:QName* in an extends *attribute information item* MUST NOT contain duplicates.[‡]

2.2.2.3 styleDefault *attribute information item*

The styleDefault *attribute information item* indicates the default style (see [2.4.1.2 Operation Style](#)) used to construct the {[element declaration](#)} properties of {[interface message references](#)} of all operations contained within the [owner element] interface.

The styleDefault *attribute information item* has the following Infoset properties:

- A [local name] of styleDefault.
- A [namespace name] which has no value.

The type of the styleDefault *attribute information item* is *list of xs:anyURI*. Its value, if present, MUST contain absolute IRIs (see [\[ETF RFC 3987\]](#)).[‡]

2.2.3 Mapping Interface's XML Representation to Component Properties

The mapping from the XML Representation of the *interface element information item* (see [2.2.2 XML Representation of Interface Component](#)) to the properties of the *Interface* component is as described in [Table 2-2](#).

Table 2-2. Mapping from XML Representation to Interface Component Properties

Property	Value
{ name }	The <i>QName</i> whose local name is actual value of the name <i>attribute information item</i> and whose namespace name is the actual value of the <i>targetNamespace attribute information item</i> of the [parent] <i>description element information item</i>
{ extended interfaces }	The set of Interface components resolved to by the values in the extends <i>attribute information item</i> , if any (see 2.17 QName resolution).
{ interface faults }	The set of Interface Fault components corresponding to the <i>fault element information items</i> in [children], if any.
{ interface operations }	The set of Interface Operation components corresponding to the <i>operation element information items</i> in [children], if any.

Recall that, per [2.2.1 The Interface Component](#), the *Interface* components in the {[extended interfaces](#)} property of a given *Interface* component MUST NOT contain that *Interface* component in any of their {[extended interfaces](#)} properties, that is to say, recursive extension of interfaces is disallowed.

2.3 Interface Fault

2.3.1 The Interface Fault Component

A fault is an event that occurs during the execution of a message exchange that disrupts the normal flow of messages.

A fault is typically raised when a party is unable to communicate an error condition inside the normal message flow, or a party wishes to terminate a message exchange. A fault message may be used to communicate out of band information such as the reason for the error, the origin of the fault, as well as other informal diagnostics such as a program stack trace.

An [Interface Fault](#) component describes a fault that MAY occur during invocation of an operation of the interface. The [Interface Fault](#) component declares an abstract fault by naming it and indicating the contents of the fault message. When and how the fault message flows is indicated by the [Interface Operation](#) component.

The [Interface Fault](#) component provides a clear mechanism to name and describe the set of faults an interface may generate. This allows operations to easily identify the individual faults they may generate by name. This mechanism allows the ready identification of the same fault occurring across multiple operations and referenced in multiple bindings as well as reducing duplication of description for an individual fault.

Faults other than the ones described in the *Interface* component may also be generated at run-time, i.e. faults are an open set. The *Interface* component describes faults that have application level semantics, i.e. that the client or

service is expected to handle, and potentially recover from, as part of the application processing logic. For example, an [Interface](#) component that accepts a credit card number may describe faults that indicate the credit card number is invalid, has been reported stolen, or has expired. The [Interface](#) component does not describe general system faults such as network failures, out of memory conditions, out of disk space conditions, invalid message formats, etc., although these faults may be generated as part of the message exchange. Such general system faults can reasonably be expected to occur in any message exchange and explicitly describing them in an [Interface](#) component is therefore uninformative.

The properties of the Interface Fault component are as follows:

- {name} REQUIRED. An *xs:QName*.
- {message content model} REQUIRED. An *xs:token* with one of the values *#any*, *#none*, *#other*, or *#element*.[‡] A value of *#any* indicates that the fault content is any single element. A value of *#none* indicates there is no fault content. A value of *#other* indicates that the fault content is described by some other extension property that references a declaration in a non-XML extension type system. A value of *#element* indicates that the fault consists of a single element described by the global element declaration referenced by the {[element declaration](#)} property. This property is used only when the fault is described using an XML-based data model.
- {element declaration} OPTIONAL. A reference to an [Element Declaration](#) component in the {[element declarations](#)} property of the [Description](#) component. This element represents the content or “payload” of the fault. When the {[message content model](#)} property has the value *#any* or *#none* the {[element declaration](#)} property MUST be empty.[‡]
- {parent} REQUIRED. The [Interface](#) component that contains this component in its {[interface faults](#)} property.

For each [Interface Fault](#) component in the {[interface faults](#)} property of an [Interface](#) component, the {[name](#)} property must be unique. Note that this constraint is enforced by the normative WSDL 2.0 XML schema.

[Interface Fault](#) components are uniquely identified by the QName of the enclosing [Interface](#) component and QName of the [Interface Fault](#) component itself.

Note:

Despite having a {[name](#)} property, [Interface Fault](#) components cannot be identified solely by their QName. Indeed, two [Interface](#) components whose {[name](#)} property value has the same namespace name, but different local names, can contain [Interface Fault](#) components with the same {[name](#)} property value. Thus, the {[name](#)} property of [Interface Fault](#) component is not sufficient to form the unique identity of an [Interface Fault](#) component. A method for uniquely identifying components is defined in [A.2 Fragment Identifiers](#). See [A.2.5 The Interface Fault Component](#) for the definition of the fragment identifier for the [Interface Fault](#) component.

In cases where, due to an interface extending one or more other interfaces, two or more [Interface Fault](#) components have the same value for their {[name](#)} property, then the component models of those [Interface Fault](#) components MUST be equivalent (see [2.15 Equivalence of Components](#)).[‡] If the [Interface Fault](#) components are equivalent then they are considered to collapse into a single component. Within the same [Interface](#) component, if two [Interface Fault](#) components are not equivalent then their {[name](#)} properties MUST NOT be equal.

Note that, due to the above rules, if two interfaces that have the same value for the namespace name of their {[name](#)} property also have one or more faults that have the same value for their {[name](#)} property, then those two interfaces cannot both form part of the derivation chain of a derived interface unless those faults are the same fault.

For the above reason, it is considered good practice to ensure, where necessary, that the local name of the {[name](#)} property of [Interface Fault](#) components within a namespace SHOULD be unique, thus allowing such derivation to occur without inadvertent error.[‡]

If a type system NOT based on the XML Infoset [[XML Information Set](#)] is in use (as considered in [3.2 Using Other Schema Languages](#)) then additional properties would need to be added to the [Interface Fault](#) component (along with extension attributes to its XML representation) to allow associating such message types with the message reference.

2.3.2 XML Representation of Interface Fault Component

```
<description>
  <interface>
    <fault
      name="xs:NCName"
      element="union of xs:QName, xs:token"? >
      <documentation />*
    </fault>
  </interface>
</description>
```

The XML representation for an [Interface Fault](#) component is an *element information item* with the following Infoset properties:

- A [local name] of `fault`
- A [namespace name] of "http://www.w3.org/ns/wsdli"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED `name` *attribute information item* as described below in [2.3.2.1 name attribute information item with fault \[owner element\]](#).

- An OPTIONAL *element attribute information item* as described below in [2.3.2.2 element attribute information item with fault \[owner element\]](#).
- Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsd1".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* (see [5. Documentation](#)).
 2. Zero or more namespace-qualified *element information item* s whose [namespace name] is NOT "http://www.w3.org/ns/wsd1 " .

2.3.2.1 name attribute information item with fault [owner element]

The name *attribute information item* identifies a given fault *element information item* inside a given interface *element information item*.

The name *attribute information item* has the following Infoset properties:

- A [local name] of name
- A [namespace name] which has no value

The type of the name *attribute information item* is *xs:NCName*.

2.3.2.2 element attribute information item with fault [owner element]

The element *attribute information item* refers, by QName, to an [Element Declaration](#) component.

The element *attribute information item* has the following Infoset properties:

- A [local name] of element.
- A [namespace name] which has no value.

The type of the element *attribute information item* is a union of *xs:QName* and *xs:token* where the allowed token values are *#any*, *#none*, or *#other*.

2.3.3 Mapping Interface Fault's XML Representation to Component Properties

The mapping from the XML Representation of the fault *element information item* (see [2.3.2 XML Representation of Interface Fault Component](#)) to the properties of the [Interface Fault](#) component is as described in [Table 2-3](#).

Table 2-3. Mapping from XML Representation to Interface Fault Component Properties

Property	Value
{name}	The QName whose local name is the actual value of the name <i>attribute information item</i> . and whose namespace name is the actual value of the targetNamespace <i>attribute information item</i> of the [parent] description <i>element information item</i> of the [parent] interface <i>element information item</i> .
{message content model}	If the element <i>attribute information item</i> is present and its value is a QName, then <i>#element</i> ; otherwise the actual value of the element <i>attribute information item</i> , if any; otherwise <i>#other</i> .
{element declaration}	If the element <i>attribute information item</i> is present and its value is a QName, then the Element Declaration component from the {element declarations} property of the Description component resolved to by the value of the element <i>attribute information item</i> (see 2.17 QName resolution); otherwise empty. If the element <i>attribute information item</i> has a value, then it MUST resolve to an Element Declaration component from the {element declarations} property of the Description component. [†]
{parent}	The Interface component corresponding to the interface <i>element information item</i> in [parent].

2.4 Interface Operation

2.4.1 The Interface Operation Component

An [Interface Operation](#) component describes an operation that a given interface supports. An operation is an interaction with the service consisting of a set of (ordinary and fault) messages exchanged between the service and the other parties involved in the interaction. The sequencing and cardinality of the messages involved in a particular interaction is governed by the *message exchange pattern* used by the operation (see [{message exchange pattern}](#) property).

A message exchange pattern defines placeholders for messages, the participants in the pattern (i.e., the sources and sinks of the messages), and the cardinality and sequencing of messages exchanged by the participants. The message placeholders are associated with specific message types by the operation that uses the pattern by means of message and fault references (see [{interface message references}](#) and [{interface fault references}](#) properties).

The service whose operation is using the pattern becomes one of the participants of the pattern. This specification does not define a machine understandable language for defining message exchange patterns, nor does it define any specific patterns. The companion specification, [\[WSDL 2.0 Adjuncts\]](#) defines a set of such patterns and defines identifying IRIs any of which MAY be used as the value of the `{message exchange pattern}` property.

The properties of the Interface Operation component are as follows:

- `{name}` REQUIRED. An `xs:QName`.
- `{message exchange pattern}` REQUIRED. An `xs:anyURI` identifying the message exchange pattern used by the operation. This `xs:anyURI` MUST be an absolute IRI (see [\[IETF RFC 3987\]](#)).[‡]
- `{interface message references}` OPTIONAL. A set of [Interface Message Reference](#) components for the ordinary messages the operation accepts or sends.
- `{interface fault references}` OPTIONAL. A set of [Interface Fault Reference](#) components for the fault messages the operation accepts or sends.
- `{style}` OPTIONAL. A set of `xs:anyURIs` identifying the rules that were used to construct the `{element declaration}` properties of `{interface message references}`. (See [2.4.1.2 Operation Style](#).) These `xs:anyURIs` MUST be absolute IRIs (see [\[IETF RFC 3986\]](#)).[‡]
- `{parent}` REQUIRED. The [Interface](#) component that contains this component in its `{interface operations}` property.

For each [Interface Operation](#) component in the `{interface operations}` property of an [Interface](#) component, the `{name}` property MUST be unique. Note that this constraint is enforced by the normative WSDL 2.0 XML schema. [Interface Operation](#) components are uniquely identified by the QName of the enclosing [Interface](#) component and QName of the [Interface Operation](#) component itself.

Note:

Despite having a `{name}` property, [Interface Operation](#) components cannot be identified solely by their QName. Indeed, two [Interface](#) components whose `{name}` property value has the same namespace name, but different local names, can contain [Interface Operation](#) components with the same `{name}` property value. Thus, the `{name}` property of [Interface Operation](#) components is not sufficient to form the unique identity of an [Interface Operation](#) component. A method for uniquely identifying components is defined in [A.2 Fragment Identifiers](#). See [A.2.6 The Interface Operation Component](#) for the definition of the fragment identifier for the [Interface Operation](#) component.

In cases where, due to an interface extending one or more other interfaces, two or more [Interface Operation](#) components have the same value for their `{name}` property, then the component models of those [Interface Operation](#) components MUST be equivalent (see [2.15 Equivalence of Components](#)).[‡] If the [Interface Operation](#) components are equivalent then they are considered to collapse into a single component. Within the same [Interface](#) component, if two [Interface Operation](#) components are not equivalent then their `{name}` properties MUST NOT be equal.

Note that, due to the above rules, if two interfaces that have the same value for the namespace name of their `{name}` property also have one or more operations that have the same value for their `{name}` property, then those two interfaces cannot both form part of the derivation chain of a derived interface unless those operations are the same operation.

For the above reason, it is considered good practice to ensure, where necessary, that the `{name}` property of [Interface Operation](#) components within a namespace SHOULD be unique, thus allowing such derivation to occur without inadvertent error.[‡]

More than one [Interface Fault Reference](#) component in the `{interface fault references}` property of an [Interface Operation](#) component may refer to the same message label. In that case, the listed fault types define alternative fault messages. This allows one to indicate that there is more than one type of fault that is related to that message.

2.4.1.1 Message Exchange Pattern

This section describes some aspects of message exchange patterns in more detail. Refer to the *Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts* specification [\[WSDL 2.0 Adjuncts\]](#) for a complete discussion of the semantics of message exchange patterns in general, as well as the definitions of the message exchange patterns that are predefined by WSDL 2.0.

A *placeholder message* is a template for an actual message as described by an [Interface Message Reference](#) component. Although a placeholder message is not itself a component, it is useful to regard it as having both a `{message label}` and a `{direction}` property which define the values of the actual [Interface Message Reference](#) component that corresponds to it. A placeholder message is also associated with some node that exchanges the message with the service. Furthermore, a placeholder message may be designated as optional in the exchange.

A *fault propagation ruleset* specifies the relation between the [Interface Fault Reference](#) and [Interface Message Reference](#) components of an [Interface Operation](#) component. The *Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts* specification [\[WSDL 2.0 Adjuncts\]](#) defines three fault propagation rulesets which we will refer to as *fault-replaces-message*, *message-triggers-fault*, and *no-faults*. These three fault propagation rulesets are used by the predefined message exchange patterns defined in [\[WSDL 2.0 Adjuncts\]](#). Other message exchange patterns can define additional fault propagation rulesets.

A *message exchange pattern* is a template for the exchange of one or more messages, and their associated faults, between the service and one or more other nodes as described by an [Interface Operation](#) component. The service and the other nodes are referred to as the *participants* in the exchange. More specifically, a message exchange

pattern consists of a sequence of one or more placeholder messages. Each placeholder message within this sequence is uniquely identified by its `{message label}` property. A message exchange pattern is itself uniquely identified by an absolute IRI, which is used as the value of the `{message exchange pattern}` property of the [Interface Operation](#) component, and which specifies the fault propagation ruleset that its faults obey.[‡]

2.4.1.2 Operation Style

An operation style specifies additional information about an operation. For example, an operation style may define structural constraints on the element declarations of the interface message reference or interface fault components used by the operation. This additional information in no way affects the messages and faults exchanged with the service and it can therefore be safely ignored in that context. However, the additional information can be used for other purposes, for example, improved code generation. The `{style}` property of the [Interface Operation](#) component contains a set of zero or more IRIs that identify operation styles. An [Interface Operation](#) component MUST satisfy the specification defined by each operation style identified by its `{style}` property. [‡] If no [Interface Operation](#) component can simultaneously satisfy all of the styles, the document is invalid.

If the `{style}` property of an [Interface Operation](#) component does have a value, then that value (a set of IRIs) specifies the rules that were used to define the element declarations (or other properties that define the message and fault contents; see [3.2 Using Other Schema Languages](#)) of the [Interface Message Reference](#) or [Interface Fault](#) components used by the operation. Although a given operation style has the ability to constrain *all* input and output messages and faults of an operation, it MAY choose to constrain any combination thereof, e.g. only the messages, or only the inputs.

Please refer to the *Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts* specification [\[WSDL 2.0 Adjuncts\]](#) for particular operation style definitions.

2.4.2 XML Representation of Interface Operation Component

```
<description>
  <interface>
    <operation
      name="xs:NCName"
      pattern="xs:anyURI"?
      style="List of xs:anyURI"? >
      <documentation />*
      [ <input /> | <output /> | <infault /> | <outfault /> ]*
    </operation>
  </interface>
</description>
```

The XML representation for an [Interface Operation](#) component is an *element information item* with the following Infoset properties:

- A [local name] of operation
- A [namespace name] of "http://www.w3.org/ns/wsdl"
- Two or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *name attribute information item* as described below in [2.4.2.1 name attribute information item with operation \[owner element\]](#).
 - An OPTIONAL *pattern attribute information item* as described below in [2.4.2.2 pattern attribute information item with operation \[owner element\]](#).
 - An OPTIONAL *style attribute information item* as described below in [2.4.2.3 style attribute information item with operation \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- One or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).
 2. One or more *element information items* from among the following, in any order:
 - One or more *element information items* from among the following, in any order:
 - Zero or more *input element information items* (see [2.5.2 XML Representation of Interface Message Reference Component](#)).
 - Zero or more *output element information items* (see [2.5.2 XML Representation of Interface Message Reference Component](#)).
 - Zero or more *infault element information items* (see [2.6.2 XML Representation of Interface Fault Reference](#)).
 - Zero or more *outfault element information items* (see [2.6.2 XML Representation of Interface Fault Reference](#)).
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".

2.4.2.1 name attribute information item with operation [owner element]

The name *attribute information item* identifies a given operation *element information item* inside a given interface *element information item*.

The name *attribute information item* has the following Infoset properties:

- A [local name] of name
- A [namespace name] which has no value

The type of the name *attribute information item* is *xs:NCName*.

2.4.2.2 pattern *attribute information item* with operation [owner element]

The pattern *attribute information item* identifies the message exchange pattern a given operation uses.

The pattern *attribute information item* has the following Infoset properties:

- A [local name] of pattern
- A [namespace name] which has no value

The type of the pattern *attribute information item* is *xs:anyURI*. Note that its value must be an absolute IRI (see [IETF RFC 3987]).

2.4.2.3 style *attribute information item* with operation [owner element]

The style *attribute information item* indicates the rules that were used to construct the {[element declaration](#)} properties of the [Interface Message Reference](#) components which are members of the {[interface message references](#)} property of the [owner element] operation.

The style *attribute information item* has the following Infoset properties:

- A [local name] of style
- A [namespace name] which has no value

The type of the style *attribute information item* is *list of xs:anyURI*. Note that its value must be an absolute IRI (see [IETF RFC 3987]).

2.4.3 Mapping Interface Operation's XML Representation to Component Properties

The mapping from the XML Representation of the operation *element information item* (see [2.4.2 XML Representation of Interface Operation Component](#)) to the properties of the Interface Operation component (see [2.4.1 The Interface Operation Component](#)) is as described in [Table 2-4](#).

Table 2-4. Mapping from XML Representation to Interface Operation Component Properties

Property	Value
{ name }	The QName whose local name is the actual value of the name <i>attribute information item</i> and whose namespace name is the actual value of the targetNamespace <i>attribute information item</i> of the [parent] description <i>element information item</i> of the [parent] interface <i>element information item</i> .
{ message exchange pattern }	The actual value of the pattern <i>attribute information item</i> ; otherwise 'http://www.w3.org/ns/wsdli/in-out'.
{ interface message references }	The set of message references corresponding to the input and output <i>element information items</i> in [children], if any.
{ interface fault references }	The set of interface fault references corresponding to the infault and outfault <i>element information items</i> in [children], if any.
{ style }	The set containing the IRIs in the actual value of the style <i>attribute information item</i> , if present; otherwise the set containing the IRIs in the actual value of the styleDefault <i>attribute information item</i> of the [parent] interface <i>element information item</i> , if present; otherwise empty.
{ parent }	The Interface component corresponding to the interface <i>element information item</i> in [parent].

2.5 Interface Message Reference

2.5.1 The Interface Message Reference Component

An [Interface Message Reference](#) component defines the content, or *payload*, of a message exchanged in an operation. By default, the message content is defined by an XML-based type system such as XML Schema. Other type systems may be used via the WSDL 2.0 type system extension mechanism.

A message exchange pattern defines a set of placeholder messages that participate in the pattern and assigns them unique message labels within the pattern (e.g. 'In', 'Out'). The purpose of an [Interface Message Reference](#) component is to associate an actual message element (XML element declaration or some other declaration (see [3.2 Using Other Schema Languages](#))) with a message in the pattern, as identified by its message label. Later, when the message exchange pattern is instantiated, messages corresponding to that particular label will follow the element assignment made by the [Interface Message Reference](#) component.

The properties of the Interface Message Reference component are as follows:

- {message label} REQUIRED. An *xs:NCName*. This property identifies the role this message plays in the {message exchange pattern} of the [Interface Operation](#) component this message is contained within. The value of this property MUST match the name of a placeholder message defined by the message exchange pattern.[‡]
- {direction} REQUIRED. An *xs:token* with one of the values *in* or *out*, indicating whether the message is coming to the service or going from the service, respectively.[‡] The direction MUST be the same as the direction of the message identified by the {message label} property in the {message exchange pattern} of the [Interface Operation](#) component this is contained within.[‡]
- {message content model} REQUIRED. An *xs:token* with one of the values *#any*, *#none*, *#other*, or *#element*.[‡] A value of *#any* indicates that the message content is any single element. A value of *#none* indicates there is no message content. A value of *#other* indicates that the message content is described by some other extension property that references a declaration in a non-XML extension type system. A value of *#element* indicates that the message consists of a single element described by the global element declaration referenced by the {element declaration} property. This property is used only when the message is described using an XML-based data model.
- {element declaration} OPTIONAL. A reference to an [Element Declaration](#) component in the {element declarations} property of the [Description](#) component. This element represents the content or "payload" of the message. When the {message content model} property has the value *#any* or *#none*, the {element declaration} property MUST be empty.[‡]
- {parent} REQUIRED. The [Interface Operation](#) component that contains this component in its {interface message references} property.

For each [Interface Message Reference](#) component in the {interface message references} property of an [Interface Operation](#) component, its {message label} property MUST be unique.[‡]

If a type system not based upon the XML Infoset is in use (as considered in [3.2 Using Other Schema Languages](#)), then additional properties would need to be added to the [Interface Message Reference](#) component (along with extension attributes to its XML representation) to allow associating such message types with the message reference.

2.5.2 XML Representation of Interface Message Reference Component

```

<description>
  <interface>
    <operation>
      <input
        messageLabel="xs:NCName"?
        element="union of xs:QName, xs:token"? >
        <documentation /*>
      </input>
      <output
        messageLabel="xs:NCName"?
        element="union of xs:QName, xs:token"? >
        <documentation /*>
      </output>
    </operation>
  </interface>
</description>

```

The XML representation for an [Interface Message Reference](#) component is an *element information item* with the following Infoset properties:

- A [local name] of input or output
- A [namespace name] of "http://www.w3.org/ns/wsdli"
- Zero or more *attribute information items* amongst its [attributes] as follows:
 - An OPTIONAL *messageLabel attribute information item* as described below in [2.5.2.1 messageLabel attribute information item with input or output \[owner element\]](#).
 - An OPTIONAL *element attribute information item* as described below in [2.5.2.2 element attribute information item with input or output \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdli".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).
 2. Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdli".

2.5.2.1 *messageLabel attribute information item with input or output [owner element]*

The *messageLabel attribute information item* identifies the role of this message in the message exchange pattern of the given operation *element information item*.

The *messageLabel attribute information item* has the following Infoset properties:

- A [local name] of *messageLabel*
- A [namespace name] which has no value

The type of the *messageLabel attribute information item* is *xs:NCName*.

2.5.2.2 *element attribute information item with input or output [owner element]*

The *element attribute information item* has the following Infoset properties:

- A [local name] of *element*.
- A [namespace name] which has no value.

The type of the *element attribute information item* is a union of *xs:QName* and *xs:token* where the allowed token values are *#any*, *#none*, or *#other*.

2.5.3 Mapping Interface Message Reference's XML Representation to Component Properties

The mapping from the XML Representation of the interface message reference *element information item* (see [2.5.2 XML Representation of Interface Message Reference Component](#)) to the properties of the [Interface Message Reference](#) component (see [2.5.1 The Interface Message Reference Component](#)) is as described in [Table 2-5](#) and uses the definitions below.

Define the *message exchange pattern* of the *element information item* to be the [{message exchange pattern}](#) of the parent [Interface Operation](#) component.

Define the *message direction* of the *element information item* to be *in* if its local name is *input*, and *out* if its local name is *output*.

Note that the *messageLabel attribute information item* of an interface message reference *element information item* must be present if the message exchange pattern has more than one placeholder message with [{direction}](#) equal to the message direction.

If the *messageLabel attribute information item* of an interface message reference *element information item* is present, then its actual value MUST match the [{message label}](#) of some placeholder message with [{direction}](#) equal to the message direction. [‡]

If the *messageLabel attribute information item* of an interface message reference *element information item* is absent then there MUST be a unique placeholder message with [{direction}](#) equal to the message direction. [‡]

Define the *effective message label* of an interface message reference *element information item* to be either the actual value of the *messageLabel attribute information item* if it is present, or the [{message label}](#) of the unique placeholder message with [{direction}](#) equal to the message direction if the *attribute information item* is absent.

If the local name is *input* then the message exchange pattern MUST have at least one placeholder message with direction "In". [‡]

If the local name is *output* then the message exchange pattern MUST have at least one placeholder message with direction "Out". [‡]

If the local name is *infault* then the message exchange pattern MUST support at least one fault in the "In" direction. [‡]

If the local name is *outfault* then the message exchange pattern MUST support at least one fault in the "Out" direction. [‡]

Table 2-5. Mapping from XML Representation to Interface Message Reference Component Properties

Property	Value
{message label}	The effective message label.
{direction}	The message direction.
{message content model}	If the <i>element attribute information item</i> is present and its value is a <i>QName</i> , then <i>#element</i> ; otherwise the actual value of the <i>element attribute information item</i> , if any; otherwise <i>#other</i> .
{element declaration}	If the <i>element attribute information item</i> is present and its value is a <i>QName</i> , then the Element Declaration component from the {element declarations} property of the Description component resolved to by the value of the <i>element attribute information item</i> (see 2.17 QName resolution); otherwise empty. If the <i>element attribute information item</i> has a value, then it MUST resolve to an Element Declaration component from the {element declarations} property of the Description component. [‡]
{parent}	The Interface Operation component corresponding to the <i>interface element information item</i> in [parent].

2.6 Interface Fault Reference

2.6.1 The Interface Fault Reference Component

An [Interface Fault Reference](#) component associates a defined type, specified by an [Interface Fault](#) component, to a fault message exchanged in an operation.

A message exchange pattern defines a set of placeholder messages that participate in the pattern and assigns them unique message labels within the pattern (e.g. 'In', 'Out'). The purpose of an [Interface Fault Reference](#) component is to associate an actual message type (XML element declaration or some other declaration (see [3.2 Using Other Schema Languages](#)) for message content, as specified by an [Interface Fault](#) component) with a fault message occurring in the pattern. In order to identify the fault message it describes, the [Interface Fault Reference](#) component uses the message label of the message the fault is associated with, as a key.

As indicated earlier, the companion specification [[WSDL 2.0 Adjuncts](#)] defines several *fault propagation rulesets* that a given message exchange pattern may use. For the ruleset *fault-replaces-message*, the message that the fault relates to identifies the message *in place of which* the declared fault message will occur. Thus, the fault message will travel in the *same* direction as the message it replaces in the pattern. For the ruleset *message-triggers-fault*, the message that the fault relates to identifies the message *after which* the indicated fault may occur, in the opposite direction of the referred to message. That is, the fault message will travel in the *opposite* direction of the message it comes after in the message exchange pattern.

The properties of the Interface Fault Reference component are as follows:

- {interface fault} REQUIRED. An [Interface Fault](#) component in the {[interface faults](#)} property of the [parent] [Interface Operation](#) component's [parent] [Interface](#) component, or an [Interface](#) component that it directly or indirectly extends. Identifying the [Interface Fault](#) component therefore indirectly defines the actual content or payload of the fault message.
- {message label} REQUIRED. An *xs:NCName*. This property identifies the message this fault relates to among those defined in the {[message exchange pattern](#)} property of the [Interface Operation](#) component it is contained within. The value of this property MUST match the name of a placeholder message defined by the message exchange pattern.[†]
- {direction} REQUIRED. A *xs:token* with one of the values *in* or *out*, indicating whether the fault is coming to the service or going from the service, respectively. The direction MUST be consistent with the direction implied by the fault propagation ruleset used in the message exchange pattern of the operation.[†] For example, if the ruleset *fault-replaces-message* is used, then a fault that refers to an outgoing message would have a {[direction](#)} property value of *out*. On the other hand, if the ruleset *message-triggers-fault* is used, then a fault that refers to an outgoing message would have a {[direction](#)} property value of *in* as the fault travels in the opposite direction of the message.
- {parent} REQUIRED. The [Interface Operation](#) component that contains this component in its {[interface fault references](#)} property.

For each [Interface Fault Reference](#) component in the {[interface fault references](#)} property of an [Interface Operation](#) component, the combination of its {[interface fault](#)} and {[message label](#)} properties MUST be unique.[†]

2.6.2 XML Representation of Interface Fault Reference

```
<description>
  <interface>
    <operation>
      <infault
        ref="xs:QName"
        messageLabel="xs:NCName"? >
        <documentation />*
      </infault>*
      <outfault
        ref="xs:QName"
        messageLabel="xs:NCName"? >
        <documentation />*
      </outfault>*
    </operation>
  </interface>
</description>
```

The XML representation for an [Interface Fault Reference](#) component is an *element information item* with the following Infoset properties:

- A [local name] of *infault* or *outfault*
- A [namespace name] of "http://www.w3.org/ns/wsdli"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *ref attribute information item* as described below in [2.6.2.1 ref attribute information item with infault, or outfault \[owner element\]](#).

- An OPTIONAL *messageLabel attribute information item* as described below in [2.6.2.2 messageLabel attribute information item with infault, or outfault \[owner element\]](#).
- Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* (see [5. Documentation](#)).
 2. Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".

2.6.2.1 *ref attribute information item with infault, or outfault [owner element]*

The *ref attribute information item* refers to a fault component.

The *ref attribute information item* has the following Infoset properties:

- A [local name] of *ref*
- A [namespace name] which has no value

The type of the *ref attribute information item* is *xs:QName*.

2.6.2.2 *messageLabel attribute information item with infault, or outfault [owner element]*

The *messageLabel attribute information item* identifies the message in the message exchange pattern of the given operation *element information item* that is associated with this fault.

The *messageLabel attribute information item* has the following Infoset properties:

- A [local name] of *messageLabel*
- A [namespace name] which has no value

The type of the *messageLabel attribute information item* is *xs:NCName*.

The *messageLabel attribute information item* MUST be present in the XML representation of an [Interface Fault Reference](#) component with a given {*direction*}, if the {*message exchange pattern*} of the parent [Interface Operation](#) component has more than one fault with that direction.[‡] Recall that the *fault propagation ruleset* of the {*message exchange pattern*} specifies the relation between faults and messages. For example, the *fault-replaces-message* ruleset specifies that the faults have the same direction as the messages, while the *message-triggers-fault* ruleset specifies that the faults have the opposite direction from the messages.

2.6.3 Mapping Interface Fault Reference's XML Representation to Component Properties

The mapping from the XML Representation of the message reference *element information item* (see [2.6.2 XML Representation of Interface Fault Reference](#)) to the properties of the Interface Fault Reference component (see [2.6.1 The Interface Fault Reference Component](#)) is as described in [Table 2-6](#) and uses the definitions below.

Define the *message exchange pattern* of the *element information item* to be the {*message exchange pattern*} of the parent [Interface Operation](#) component.

Define the *fault direction* of the *element information item* to be *in* if its local name is *infault* and *out* if its local name is *outfault*.

Define the *message direction* of the *element information item* to be the {*direction*} of the placeholder message associated with the fault as specified by the fault propagation ruleset of the message exchange pattern.

The *messageLabel attribute information item* of an interface fault reference *element information item* MUST be present if the message exchange pattern has more than one placeholder message with {*direction*} equal to the message direction.[‡]

If the *messageLabel attribute information item* of an interface fault reference *element information item* is present then its actual value MUST match the {*message label*} of some placeholder message with {*direction*} equal to the message direction.[‡]

If the *messageLabel attribute information item* of an interface fault reference *element information item* is absent then there MUST be a unique placeholder message with {*direction*} equal to the message direction.[‡]

Define the *effective message label* of an interface fault reference *element information item* to be either the actual value of the *messageLabel attribute information item* if it is present, or the {*message label*} of the unique placeholder message whose {*direction*} is equal to the message direction if the *attribute information item* is absent.

Table 2-6. Mapping from XML Representation to Interface Fault Reference Component Properties

Property	Value
{ <i>interface fault</i> }	The Interface Fault component from { <i>interface faults</i> } property of the parent Interface component, or an Interface component that it directly or indirectly extends, with { <i>name</i> } equal to the actual value of the <i>ref attribute information item</i> .
{ <i>message label</i> }	The effective message label.

<code>{direction}</code>	The fault direction.
<code>{parent}</code>	The Interface Operation component corresponding to the <code>interface</code> <i>element information item</i> in <code>[parent]</code> .

2.7 Binding

2.7.1 The Binding Component

A [Binding](#) component describes a concrete message format and transmission protocol which may be used to define an endpoint (see [2.13 Endpoint](#)). That is, a [Binding](#) component defines the implementation details necessary to access the service.

[Binding](#) components can be used to describe such information in a reusable manner for any interface or specifically for a given interface. Furthermore, binding information MAY be specified on a per-operation basis (see [2.9.1 The Binding Operation Component](#)) within an interface, in addition to across all operations of an interface.

If a [Binding](#) component specifies any operation-specific binding details (by including [Binding Operation](#) components) or any fault binding details (by including [Binding Fault](#) components), then it MUST specify an interface the [Binding](#) component applies to, so as to indicate which interface the operations come from.[‡]

Conversely, a [Binding](#) component which omits any operation-specific binding details and any fault binding details MAY omit specifying an interface. [Binding](#) components that do not specify an interface MAY be used to specify operation-independent binding details for [Service](#) components with different interfaces. That is, such [Binding](#) components are reusable across one or more interfaces.

No concrete binding details are given in this specification. The companion specification, *Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts* [[WSDL 2.0 Adjuncts](#)] defines such bindings for SOAP 1.2 [[SOAP 1.2 Part 1: Messaging Framework \(Second Edition\)](#)] and HTTP [[IETF RFC 2616](#)]. Other specifications MAY define additional binding details. Such specifications are expected to annotate the [Binding](#) component (and its sub-components) with additional properties and specify the mapping from the XML representation to these properties.

A [Binding](#) component that defines bindings for an [Interface](#) component MUST define bindings for all the operations of that [Interface](#) component.[‡] The bindings can occur via defaulting rules which allow one to specify default bindings for all operations and faults (see, for example [[WSDL 2.0 Adjuncts](#)]) or by defining bindings for each [Interface Operation](#) and [Interface Fault](#) component of the [Interface](#) component.

Similarly, whenever a reusable [Binding](#) component (i.e. one that does not specify an [Interface](#) component) is applied to a specific [Interface](#) component in the context of an [Endpoint](#) component (see [2.13.1 The Endpoint Component](#)), the [Binding](#) component MUST define bindings for each [Interface Operation](#) and [Interface Fault](#) component of the [Interface](#) component, via a combination of properties defined on the [Binding](#) component itself and default binding rules specific to its binding type.[‡]

A [Binding](#) component that defines bindings for an [Interface](#) component MUST define bindings for all the faults of that [Interface](#) component that are referenced from any of the operations in that [Interface](#) component.[‡] As for the case of operations, the binding can be defined by defaulting rules. Note that only the faults actually referenced by operations are required to have bindings.

Bindings are named constructs and can be referred to by QName (see [2.17 QName resolution](#)). For instance, [Endpoint](#) components refer to bindings in this way.

The properties of the Binding component are as follows:

- `{name}` REQUIRED. An `xs:QName`.
- `{interface}` OPTIONAL. An [Interface](#) component indicating the interface for which binding information is being specified.
- `{type}` REQUIRED. An `xs:anyURI`. This `xs:anyURI` MUST be an absolute IRI as defined by [[IETF RFC 3987](#)].[‡] The value of this IRI indicates what kind of concrete binding details are contained within this [Binding](#) component. Specifications (such as [[WSDL 2.0 Adjuncts](#)]) that define such concrete binding details MUST specify appropriate values for this property. The value of this property MAY be the namespace name of the extension elements or attributes which define those concrete binding details.
- `{binding faults}` OPTIONAL. A set of [Binding Fault](#) components.
- `{binding operations}` OPTIONAL. A set of [Binding Operation](#) components.

For each [Binding](#) component in the `{bindings}` property of a [Description](#) component, the `{name}` property MUST be unique.[‡]

2.7.2 XML Representation of Binding Component

```
<description>
  <binding
    name="xs:NCName"
    interface="xs:QName"?
    type="xs:anyURI" >
    <documentation />*
    [ <fault /> | <operation /> ]*
```

```
</binding>
</description>
```

The XML representation for a [Binding](#) component is an *element information item* with the following Infoset properties:

- A [local name] of `binding`
- A [namespace name] of `"http://www.w3.org/ns/wsdli"`
- Two or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED `name` *attribute information item* as described below in [2.7.2.1 name attribute information item with binding \[owner element\]](#).
 - An OPTIONAL `interface` *attribute information item* as described below in [2.7.2.2 interface attribute information item with binding \[owner element\]](#).
 - An REQUIRED `type` *attribute information item* as described below in [2.7.2.3 type attribute information item with binding \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdli"`.
- Zero or more *element information items* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* (see [5. Documentation](#)).
 2. Zero or more *element information items* from among the following, in any order:
 - Zero or more `fault` *element information items* (see [2.8.2 XML Representation of Binding Fault Component](#)).
 - Zero or more `operation` *element information items* (see [2.9.2 XML Representation of Binding Operation Component](#)).
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdli"`. Such *element information items* are considered to be binding extension elements (see [2.7.2.4 Binding extension elements](#)).

2.7.2.1 name attribute information item with binding [owner element]

The `name` *attribute information item* together with the `targetNamespace` *attribute information item* of the *description element information item* forms the QName of the binding.

The `name` *attribute information item* has the following Infoset properties:

- A [local name] of `name`
- A [namespace name] which has no value

The type of the `name` *attribute information item* is `xs:NCName`.

2.7.2.2 interface attribute information item with binding [owner element]

The `interface` *attribute information item* refers, by QName, to an [Interface](#) component.

The `interface` *attribute information item* has the following Infoset properties:

- A [local name] of `interface`
- A [namespace name] which has no value

The type of the `interface` *attribute information item* is `xs:QName`.

2.7.2.3 type attribute information item with binding [owner element]

The `type` *attribute information item* identifies the kind of binding details contained in the [Binding](#) component.

The `type` *attribute information item* has the following Infoset properties:

- A [local name] of `type`
- A [namespace name] which has no value

The type of the `type` *attribute information item* is `xs:anyURI`.

2.7.2.4 Binding extension elements

Binding extension elements are used to provide information specific to a particular binding. The semantics of such *element information items* are defined by the specification for those *element information items*. Such specifications are expected to annotate the [Binding](#) component with additional properties and specify the mapping from the XML representation to those properties.

2.7.3 Mapping Binding's XML Representation to Component Properties

The mapping from the XML Representation of the `binding` *element information item* (see [2.7.2 XML Representation of Binding Component](#)) to the properties of the `Binding` component (see [2.7.1 The Binding Component](#)) is as described in [Table 2-7](#).

Table 2-7. Mapping from XML Representation to Binding Component Properties

Property	Value
<code>{name}</code>	The QName whose local name is the actual value of the <code>name</code> <i>attribute information item</i> and whose namespace name is the actual value of the <code>targetNamespace</code> <i>attribute information item</i> of the [parent] <i>description element information item</i> .
<code>{interface}</code>	The <code>Interface</code> component resolved to by the actual value of the <code>interface</code> <i>attribute information item</i> (see 2.17 QName resolution), if any.
<code>{type}</code>	The actual value of the <code>type</code> <i>attribute information item</i> .
<code>{binding faults}</code>	The set of <code>Binding Fault</code> components corresponding to the <code>fault</code> <i>element information items</i> in [children], if any.
<code>{binding operations}</code>	The set of <code>Binding Operation</code> components corresponding to the <code>operation</code> <i>element information items</i> in [children], if any.

2.8 Binding Fault

2.8.1 The Binding Fault Component

A `Binding Fault` component describes a concrete binding of a particular fault within an interface to a particular concrete message format. A particular fault of an interface is uniquely identified by its `{name}` property.

Note that the fault does not occur by itself -it occurs as part of a message exchange as defined by an `Interface Operation` component (and its binding counterpart the `Binding Operation` component). Thus, the fault binding information specified in a `Binding Fault` component describes how faults that occur within a message exchange of an operation will be formatted and carried in the transport.

The properties of the Binding Fault component are as follows:

- `{interface fault}` REQUIRED. An `Interface Fault` component in the `{interface faults}` property of the `Interface` component identified by the `{interface}` property of the parent `Binding` component, or an `Interface` component that that `Interface` component directly or indirectly extends. This is the `Interface Fault` component for which binding information is being specified.
- `{parent}` REQUIRED. The `Binding` component that contains this component in its `{binding faults}` property.

For each `Binding Fault` component in the `{binding faults}` property of a `Binding` component, the `{interface fault}` property MUST be unique.¹ That is, one cannot define multiple bindings for the same fault within a given `Binding` component.

2.8.2 XML Representation of Binding Fault Component

```
<description>
  <binding>
    <fault
      ref="xs:QName" >
      <documentation />*
    </fault>
  </binding>
</description>
```

The XML representation for a `Binding Fault` component is an *element information item* with the following Infoset properties:

- A [local name] of `fault`
- A [namespace name] of "http://www.w3.org/ns/wsdl"
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *ref* *attribute information item* as described below in [2.8.2.1 ref attribute information item with fault \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).
 2. Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl". Such *element information items* are considered to be binding fault extension elements as described further below (see [2.8.2.2 Binding Fault extension elements](#)).

2.8.2.1 *ref* attribute information item with *fault* [owner element]

The *ref* attribute information item has the following Infoset properties:

- A [local name] of *ref*
- A [namespace name] which has no value

The type of the *ref* attribute information item is *xs:QName*.

2.8.2.2 Binding Fault extension elements

Binding Fault extension elements are used to provide information specific to a particular fault in a binding. The semantics of such *element information items* are defined by the specification for those *element information items*. Such specifications are expected to annotate the [Binding Fault](#) component with additional properties and specify the mapping from the XML representation to those properties.

2.8.3 Mapping Binding Fault's XML Representation to Component Properties

The mapping from the XML Representation of the *fault* *element information item* (see [2.8.2 XML Representation of Binding Fault Component](#)) to the properties of the [Binding Fault](#) component (see [2.8.1 The Binding Fault Component](#)) is as described in [Table 2-8](#).

Table 2-8. Mapping from XML Representation to Binding Fault Component Properties

Property	Value
{ interface fault }	The Interface Fault component corresponding to the actual value of the <i>ref</i> attribute information item.
{ parent }	The Binding component corresponding to the <i>binding</i> <i>element information item</i> in [parent].

2.9 Binding Operation

2.9.1 The Binding Operation Component

The [Binding Operation](#) component describes the concrete message format(s) and protocol interaction(s) associated with a particular interface operation for a given endpoint. A particular operation of an interface is uniquely identified by its [name](#) property.

The properties of the Binding Operation component are as follows:

- {[interface operation](#)} REQUIRED. An [Interface Operation](#) component in the {[interface operations](#)} property of the [Interface](#) component identified by the {[interface](#)} property of the [parent] [Binding](#) component, or an [Interface](#) component that that [Interface](#) component directly or indirectly extends. This is the [Interface Operation](#) component for which binding information is being specified.
- {[binding message references](#)} OPTIONAL. A set of [Binding Message Reference](#) components.
- {[binding fault references](#)} OPTIONAL. A set of [Binding Fault Reference](#) components.
- {[parent](#)} REQUIRED. The [Binding](#) component that contains this component in its {[binding operations](#)} property.

For each [Binding Operation](#) component in the {[binding operations](#)} property of a [Binding](#) component, the {[interface operation](#)} property MUST be unique.[†] That is, one cannot define multiple bindings for the same operation within a given [Binding](#) component.

2.9.2 XML Representation of Binding Operation Component

```
<description>
  <binding>
    <operation
      ref="xs:QName" >
      <documentation />*
      [ <input /> | <output /> | <infault /> | <outfault /> ]*
    </operation>
  </binding>
</description>
```

The XML representation for a [Binding Operation](#) component is an *element information item* with the following Infoset properties:

- A [local name] of *operation*
- A [namespace name] of "http://www.w3.org/ns/wsdli"
- One or more *attribute information items* amongst its [attributes] as follows:

- A REQUIRED *ref attribute information item* as described below in [2.9.2.1 ref attribute information item with operation \[owner element\]](#).
- Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information items* amongst its [children], in order, as follows:
 1. Zero or more documentation *element information items* (see [5. Documentation](#)).
 2. Zero or more *element information items* from among the following, in any order:
 - Zero or more input *element information items* (see [2.10 Binding Message Reference](#))
 - Zero or more output *element information items* (see [2.10 Binding Message Reference](#))
 - Zero or more infault *element information items* (see [2.11 Binding Fault Reference](#))
 - Zero or more outfault *element information items* (see [2.11 Binding Fault Reference](#))
 - Zero or more namespace-qualified *element information item* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl ". Such *element information items* are considered to be binding operation extension elements as described below (see [2.9.2.2 Binding Operation extension elements](#)).

2.9.2.1 *ref attribute information item with operation [owner element]*

The *ref attribute information item* has the following Infoset properties:

- A [local name] of *ref*
- A [namespace name] which has no value

The type of the *ref attribute information item* is *xs:QName*.

2.9.2.2 Binding Operation extension elements

Binding Operation extension elements are used to provide information specific to a particular operation in a binding. The semantics of such *element information items* are defined by the specification for those *element information items*. Such specifications are expected to annotate the [Binding Operation](#) component with additional properties and specify the mapping from the XML representation to those properties.

2.9.3 Mapping Binding Operation's XML Representation to Component Properties

The mapping from the XML Representation of the *operation element information item* (see [2.9.2 XML Representation of Binding Operation Component](#)) to the properties of the [Binding Operation](#) component is as described in [Table 2-9](#).

Table 2-9. Mapping from XML Representation to Binding Operation Component Properties

Property	Value
{ interface operation }	The Interface Operation component corresponding to the actual value of the <i>ref attribute information item</i> .
{ binding message references }	The set of Binding Message Reference components corresponding to the input and output <i>element information items</i> in [children], if any.
{ binding fault references }	The set of Binding Fault Reference components corresponding to the infault and outfault <i>element information items</i> in [children], if any.
{ parent }	The Binding component corresponding to the <i>binding element information item</i> in [parent].

2.10 Binding Message Reference

2.10.1 The Binding Message Reference Component

A [Binding Message Reference](#) component describes a concrete binding of a particular message participating in an operation to a particular concrete message format.

The properties of the Binding Message Reference component are as follows:

- {[interface message reference](#)} REQUIRED. An [Interface Message Reference](#) component among those in the {[interface message references](#)} property of the [Interface Operation](#) component being bound by the containing [Binding Operation](#) component.
- {[parent](#)} REQUIRED. The [Binding Operation](#) component that contains this component in its {[binding message references](#)} property.

For each [Binding Message Reference](#) component in the {[binding message references](#)} property of a [Binding Operation](#) component, the {[interface message reference](#)} property MUST be unique.[†] That is, the same message cannot be bound twice within the same operation.

2.10.2 XML Representation of Binding Message Reference Component

```

<description>
  <binding>
    <operation>
      <input
        messageLabel="xs:NCName"? >
        <documentation />*
      </input>
      <output
        messageLabel="xs:NCName"? >
        <documentation />*
      </output>
    </operation>
  </binding>
</description>

```

The XML representation for a [Binding Message Reference](#) component is an *element information item* with the following Infoset properties:

- A [local name] of input or output.
- A [namespace name] of "http://www.w3.org/ns/wsdl".
- Zero or more *attribute information items* amongst its [attributes] as follows:
 - An OPTIONAL `messageLabel` *attribute information item* as described below in [2.10.2.1 messageLabel attribute information item with input or output \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more `documentation` *element information items* (see [5. Documentation](#)).
 2. Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl". Such *element information items* are considered to be binding message reference extension elements as described below (see [2.10.2.2 Binding Message Reference extension elements](#)).

2.10.2.1 messageLabel attribute information item with input or output [owner element]

The `messageLabel` *attribute information item* has the following Infoset properties:

- A [local name] of `messageLabel`.
- A [namespace name] which has no value.

The type of the `messageLabel` *attribute information item* is `xs:NCName`.

2.10.2.2 Binding Message Reference extension elements

Binding Message Reference extension elements are used to provide information specific to a particular message in an operation. The semantics of such *element information items* are defined by the specification for those *element information items*. Such specifications are expected to annotate the [Binding Message Reference](#) component with additional properties and specify the mapping from the XML representation to those properties.

2.10.3 Mapping Binding Message Reference's XML Representation to Component Properties

The mapping from the XML Representation of the `binding` *element information item* (see [2.10.2 XML Representation of Binding Message Reference Component](#)) to the properties of the [Binding Message Reference](#) component is as described in [Table 2-10](#) and uses the definitions below.

Define the *message exchange pattern* of the *element information item* to be the [{message exchange pattern}](#) of the [Interface Operation](#) component being bound.

Define the *message direction* of the *element information item* to be *in* if its local name is `input` and *out* if its local name is `output`.

Note that the `messageLabel` *attribute information item* of a binding message reference *element information item* must be present if the message exchange pattern has more than one placeholder message with [{direction}](#) equal to the message direction.

If the `messageLabel` *attribute information item* of a binding message reference *element information item* is present then its actual value MUST match the [{message label}](#) of some placeholder message with [{direction}](#) equal to the message direction. [†]

If the `messageLabel` *attribute information item* of a binding message reference *element information item* is absent then there MUST be a unique placeholder message with [{direction}](#) equal to the message direction. [†]

Define the *effective message label* of a binding message reference *element information item* to be either the actual value of the `messageLabel` *attribute information item* if it is present, or the [{message label}](#) of the unique placeholder

message with [{direction}](#) equal to the message direction if the *attribute information item* is absent.

Table 2-10. Mapping from XML Representation to Binding Message Reference Component Properties

Property	Value
{interface message reference}	The Interface Message Reference component in the {interface message references} of the Interface Operation component being bound with {message label} equal to the effective message label.
{parent}	The Binding Operation component corresponding to the operation <i>element information item</i> in [parent] .

2.11 Binding Fault Reference

2.11.1 The Binding Fault Reference Component

A [Binding Fault Reference](#) component describes a concrete binding of a particular fault participating in an operation to a particular concrete message format.

The properties of the Binding Fault Reference component are as follows:

- [{interface fault reference}](#) REQUIRED. An [Interface Fault Reference](#) component among those in the [{interface fault references}](#) property of the [Interface Operation](#) component being bound by the parent [Binding Operation](#) component.
- [{parent}](#) REQUIRED. The [Binding Operation](#) component that contains this component in its [{binding fault references}](#) property.

For each [Binding Fault Reference](#) component in the [{binding fault references}](#) property of a [Binding Operation](#) component, the [{interface fault reference}](#) property MUST be unique.[†] That is, the same fault cannot be bound twice within the same operation.

2.11.2 XML Representation of Binding Fault Reference Component

```
<description>
  <binding>
    <operation>
      <infault
        ref="xs:QName"
        messageLabel="xs:NCName"?>
        <documentation />*
      </infault>
      <outfault
        ref="xs:QName"
        messageLabel="xs:NCName"?>
        <documentation />*
      </outfault>
    </operation>
  </binding>
</description>
```

The XML representation for a [Binding Fault Reference](#) component is an *element information item* with the following Infoset properties:

- A [local name] of `infault` or `outfault`.
- A [namespace name] of `"http://www.w3.org/ns/wsdl"`.
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *ref attribute information item* as described below in [2.11.2.1 ref attribute information item with infault or outfault \[owner element\]](#).
An OPTIONAL *messageLabel attribute information item* as described below in [2.11.2.2 messageLabel attribute information item with infault or outfault \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"`.
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).
 2. Zero or more namespace-qualified *element information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"`. Such *element information items* are considered to be binding fault reference extension elements as described below (see [2.11.2.3 Binding Fault Reference extension elements](#)).

2.11.2.1 ref attribute information item with infault or outfault [owner element]

The *ref attribute information item* has the following Infoset properties:

- A [local name] of `ref`.

- A [namespace name] which has no value.

The type of the `ref` *attribute information item* is `xs:QName`.

2.11.2.2 `messageLabel` *attribute information item* with `infault` or `outfault` [owner element]

The `messageLabel` *attribute information item* has the following Infoset properties:

- A [local name] of `messageLabel`.
- A [namespace name] which has no value.

The type of the `messageLabel` *attribute information item* is `xs:NCName`.

2.11.2.3 Binding Fault Reference extension elements

Binding Fault Reference extension elements are used to provide information specific to a particular fault in an operation. The semantics of such *element information items* are defined by the specification for those *element information items*. Such specifications are expected to annotate the [Binding Fault Reference](#) component with additional properties and specify the mapping from the XML representation to those properties.

2.11.3 Mapping Binding Fault Reference's XML Representation to Component Properties

The mapping from the XML Representation of the `binding` *element information item* (see [2.11.2 XML Representation of Binding Fault Reference Component](#)) to the properties of the [Binding Fault Reference](#) component is as described in [Table 2-11](#) and uses the definitions below.

Define the *message exchange pattern* of the *element information item* to be the {[message exchange pattern](#)} of the [Interface Operation](#) component being bound.

Define the *fault direction* of the *element information item* to be *in* if its local name is `infault` and *out* if its local name is `outfault`.

Define the *message direction* of the *element information item* to be the {[direction](#)} of the placeholder message associated with the fault as specified by the fault propagation ruleset of the message exchange pattern.

The `messageLabel` *attribute information item* of a binding fault reference *element information item* MUST be present if the message exchange pattern has more than one placeholder message with {[direction](#)} equal to the message direction. [†]

If the `messageLabel` *attribute information item* of a binding fault reference *element information item* is present then its actual value MUST match the {[message label](#)} of some placeholder message with {[direction](#)} equal to the message direction. [†]

If the `messageLabel` *attribute information item* of a binding fault reference *element information item* is absent then there MUST be a unique placeholder message with {[direction](#)} equal to the message direction. [†]

Define the *effective message label* of a binding fault reference *element information item* to be either the actual value of the `messageLabel` *attribute information item* if it is present, or the {[message label](#)} of the unique placeholder message with {[direction](#)} equal to the message direction if the *attribute information item* is absent.

There MUST be an [Interface Fault Reference](#) component in the {[interface fault references](#)} of the [Interface Operation](#) being bound with {[message label](#)} equal to the effective message label and with {[interface fault](#)} equal to an [Interface Fault](#) component with {[name](#)} equal to the actual value of the `ref` *attribute information item*. [†]

Table 2-11. Mapping from XML Representation to Binding Fault Reference Component Properties

Property	Value
{ interface fault reference }	The Interface Fault Reference component in the { interface fault references } of the Interface Operation being bound with { message label } equal to the effective message label, and with { interface fault } equal to an Interface Fault component with { name } equal to the actual value of the <code>ref</code> <i>attribute information item</i> .
{parent}	The Binding Operation component corresponding to the operation <i>element information item</i> in [parent].

2.12 Service

2.12.1 The Service Component

A [Service](#) component describes a set of endpoints (see [2.13 Endpoint](#)) at which a particular deployed implementation of the service is provided. The endpoints thus are in effect alternate places at which the service is provided.

Services are named constructs and can be referred to by QName (see [2.17 QName resolution](#)).

The properties of the Service component are as follows:

- {name} REQUIRED. An *xs:QName*.
- {interface} REQUIRED. An [Interface](#) component.
- {endpoints} REQUIRED. A non-empty set of [Endpoint](#) components.

For each [Service](#) component in the {services} property of a [Description](#) component, the {name} property MUST be unique.[†]

2.12.2 XML Representation of Service Component

```
<description>
  <service
    name="xs:NCName"
    interface="xs:QName" >
    <documentation />*
    <endpoint />+
  </service>
</description>
```

The XML representation for a [Service](#) component is an *element information item* with the following Infoset properties:

- A [local name] of *service*
- A [namespace name] of "http://www.w3.org/ns/wsdl"
- Two or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *name attribute information item* as described below in [2.12.2.1 name attribute information item with service \[owner element\]](#).
 - A REQUIRED *interface attribute information item* as described below in [2.12.2.2 interface attribute information item with service \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- One or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).
 2. One or more *element information items* from among the following, in any order:
 - One or more *endpoint element information items* (see [2.13.2 XML Representation of Endpoint Component](#))
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".

2.12.2.1 name attribute information item with service [owner element]

The *name attribute information item* together with the *targetNamespace attribute information item* of the *description element information item* forms the *QName* of the service.

The *name attribute information item* has the following Infoset properties:

- A [local name] of *name*
- A [namespace name] which has no value

The type of the *name attribute information item* is *xs:NCName*.

2.12.2.2 interface attribute information item with service [owner element]

The *interface attribute information item* identifies the interface that the service is an instance of.

The *interface attribute information item* has the following Infoset properties:

- A [local name] of *interface*
- A [namespace name] which has no value

The type of the *interface attribute information item* is *xs:QName*.

2.12.3 Mapping Service's XML Representation to Component Properties

The mapping from the XML Representation of the *service element information item* (see [2.12.2 XML Representation of Service Component](#)) to the properties of the [Service](#) component is as described in [Table 2-12](#).

Table 2-12. Mapping from XML Representation to Service Component Properties

Property	Value
{name}	The <i>QName</i> whose local name is the actual value of the <i>name attribute information item</i> , and whose namespace name is the actual value of the <i>targetNamespace attribute information item</i>

	of the [parent] description <i>element information item</i> .
{ interface }	The Interface component resolved to by the actual value of the interface <i>attribute information item</i> (see 2.17 QName resolution).
{ endpoints }	The Endpoint components corresponding to the endpoint <i>element information items</i> in [children].

2.13 Endpoint

2.13.1 The Endpoint Component

An [Endpoint](#) component defines the particulars of a specific endpoint at which a given service is available.

[Endpoint](#) components are local to a given [Service](#) component (see [A.2 Fragment Identifiers](#)).

The [Binding](#) component specified by the {[binding](#)} property of an [Endpoint](#) component is said to be *applied* to the [Interface](#) component which is the value of the {[interface](#)} property of the parent [Service](#) component of the [Endpoint](#). According to the constraints given below, if this [Binding](#) component has an {[interface](#)} property, its value must be the [Interface](#) component the [Binding](#) component is applied to.

The {[address](#)} property is optional to allow for means other than IRIs to be used, e.g. a WS-Addressing Endpoint Reference [[WSA 1.0 Core](#)]. It is also possible that, in certain scenarios, an address will not be required, in which case this property may be absent.

The properties of the Endpoint component are as follows:

- {[name](#)} REQUIRED. An *xs:NCName*.
- {[binding](#)} REQUIRED. A [Binding](#) component.
- {[address](#)} OPTIONAL. An *xs:anyURI*. This *xs:anyURI* MUST be an absolute IRI as defined by [[IETF RFC 3987](#)].[‡] If present, the value of this attribute represents the network address at which the service indicated by the parent [Service](#) component's {[interface](#)} property is offered via the binding referred to by the {[binding](#)} property. **Note** that the presence in this property of the characters "?" and "#" can conflict with those potentially added by the query string serialization mechanism, as defined in [Serialization as "application/x-www-form-urlencoded"](#) ([[WSDL 2.0 Adjuncts](#)], section 6.8.2).
- {[parent](#)} REQUIRED. The [Service](#) component that contains this component in its {[endpoints](#)} property.

For each [Endpoint](#) component in the {[endpoints](#)} property of a [Service](#) component, the {[name](#)} property MUST be unique. Note that this constraint is enforced by the normative WSDL 2.0 XML schema.

For each [Endpoint](#) component in the {[endpoints](#)} property of a [Service](#) component, the {[binding](#)} property MUST either be a [Binding](#) component with an unspecified {[interface](#)} property or a [Binding](#) component with an {[interface](#)} property equal to the {[interface](#)} property of the [Service](#) component.[‡]

2.13.2 XML Representation of Endpoint Component

```
<description>
  <service>
    <endpoint
      name="xs:NCName"
      binding="xs:QName"
      address="xs:anyURI"? >
      <documentation />*
    </endpoint>+
  </service>
</description>
```

The XML representation for a [Endpoint](#) component is an *element information item* with the following Infoset properties:

- A [local name] of endpoint.
- A [namespace name] of "http://www.w3.org/ns/wsdl".
- Two or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *name attribute information item* as described below in [2.13.2.1 name attribute information item with endpoint \[owner element\]](#).
 - A REQUIRED *binding attribute information item* as described below in [2.13.2.2 binding attribute information item with endpoint \[owner element\]](#).
 - An OPTIONAL *address attribute information item* as described below in [2.13.2.3 address attribute information item with endpoint \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information item* amongst its [children], in order, as follows:
 1. Zero or more *documentation element information items* (see [5. Documentation](#)).

2. Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl". Such *element information items* are considered to be endpoint extension elements as described below (see [2.13.2.4 Endpoint extension elements](#)).

2.13.2.1 *name attribute information item with endpoint [owner element]*

The *name attribute information item* together with the *targetNamespace attribute information item* of the description *element information item* forms the QName of the endpoint.

The *name attribute information item* has the following Infoset properties:

- A [local name] of *name*.
- A [namespace name] which has no value.

The type of the *name attribute information item* is *xs:NCName*.

2.13.2.2 *binding attribute information item with endpoint [owner element]*

The *binding attribute information item* refers, by QName, to a [Binding](#) component

The *binding attribute information item* has the following Infoset properties:

- A [local name] of *binding*
- A [namespace name] which has no value

The type of the *binding attribute information item* is *xs:QName*.

2.13.2.3 *address attribute information item with endpoint [owner element]*

The *address attribute information item* specifies the address of the endpoint.

The *address attribute information item* has the following Infoset properties:

- A [local name] of *address*
- A [namespace name] which has no value

The type of the *address attribute information item* is *xs:anyURI*.

2.13.2.4 *Endpoint extension elements*

Endpoint extension elements are used to provide information specific to a particular endpoint in a server. The semantics of such *element information items* are defined by the specification for those *element information items*. Such specifications are expected to annotate the [Endpoint](#) component with additional properties and specify the mapping from the XML representation to those properties.

2.13.3 Mapping Endpoint's XML Representation to Component Properties

The mapping from the XML Representation of the *endpoint element information item* (see [2.13.2 XML Representation of Endpoint Component](#)) to the properties of the [Endpoint](#) component is as described in [Table 2-13](#).

Table 2-13. Mapping from XML Representation to Endpoint Component Properties

Property	Value
{ name }	The actual value of the <i>name attribute information item</i> .
{ binding }	The Binding component resolved to by the actual value of the <i>binding attribute information item</i> (see 2.17 QName resolution).
{ address }	The actual value of the <i>address attribute information item</i> if present; otherwise empty.
{ parent }	The Service component corresponding to the <i>service element information item</i> in [parent].

2.14 XML Schema 1.0 Simple Types Used in the Component Model

The XML Schema 1.0 simple types [[XML Schema: Datatypes](#)] used in this specification are:

- *xs:token*
- *xs:NCName*
- *xs:anyURI*
- *xs:QName*

- [xs:boolean](#)

2.15 Equivalence of Components

Two component instances of the same type are considered equivalent if, for each property value of the first component, there is a corresponding property with an equivalent value on the second component, and vice versa.

- For values of a simple type (see [2.14 XML Schema 1.0 Simple Types Used in the Component Model](#)) this means that they contain the same values. For instance, two string values are equivalent if they contain the same sequence of Unicode characters, as described in [\[Character Model for the WWW\]](#), or two boolean values are equivalent if they contain the same canonical value (`true` or `false`).
- Values which are references to other components are considered equivalent when they refer to equivalent components (as determined above).
- List-based values are considered equivalent if they have the same length and their elements at corresponding positions are equivalent.
- Finally, set-based values are considered equivalent if, for each value in the first, there is an equivalent value in the second, and vice versa.

Extension properties which are not string values, sets of strings or references MUST describe their values' equivalence rules.[‡]

Because different top-level components (e.g., [Interface](#), [Binding](#), and [Service](#)) are required to have different names, it is possible to determine whether two top-level components of a given type are equivalent by simply examining their {name} property.

The [Binding](#) component specified by the {binding} property of an [Endpoint](#) is said to be applied to the [Interface](#) component which is the value of the {interface} property of the {parent} [Service](#) component for the [Endpoint](#). Note that, if this [Binding](#) component has an {interface} property, then its value MUST be the [Interface](#) component that the [Binding](#) component is applied to.

2.16 Symbol Spaces

This specification defines three symbol spaces, one for each top-level component type ([Interface](#), [Binding](#) and [Service](#)).

Within a symbol space, all qualified names (that is, the {name} property) are unique. Between symbol spaces, the names need not be unique. Thus it is perfectly coherent to have, for example, a binding and an interface that have the same name.

When XML Schema is being used as one of the type systems for a WSDL 2.0 description, then six other symbol spaces also exist, one for each of: global element declarations, global attribute declarations, named model groups, named attribute groups, type definitions and key constraints, as defined by [\[XML Schema: Structures\]](#). Other type systems may define additional symbol spaces.

2.17 QName resolution

In its serialized form WSDL 2.0 makes significant use of references between components. Such references are made using the Qualified Name, or QName, of the component being referred to. QNames are a tuple, consisting of two parts; a namespace name and a local name. The namespace name for a component is represented by the value of the *targetNamespace* attribute information item of the [parent] *description* element information item. The local name is represented by the {name} property of the component.

QName references are resolved by looking in the appropriate property of the [Description](#) component. For example, to resolve a QName of an interface (as referred to by the *interface* attribute information item on a binding), the {interfaces} property of the [Description](#) component would be inspected.

If the appropriate property of the [Description](#) component does not contain a component with the required QName, then the reference is a broken reference. A [Description](#) component MUST NOT have such broken references.[‡]

2.18 Comparing URIs and IRIs

This specification uses absolute URIs and IRIs to identify several components and components characteristics (for example, operation message exchange patterns and styles). When such absolute URIs and IRIs are being compared to determine equivalence (see [2.15 Equivalence of Components](#)), they MUST be compared character-by-character as indicated in [\[IETF RFC 3987\]](#).[‡]

3. Types

```
<description>
  <types>
    <documentation />*
    [ <xs:import namespace="xs:anyURI" schemaLocation="xs:anyURI"? /> |
      <xs:schema targetNamespace="xs:anyURI"? /> |
      other extension elements ]*
  </types>
</description>
```

The content of messages and faults may be constrained using type system components. These constraints are based upon a specific data model, and expressed using a particular schema language.

Although a variety of data models can be accommodated (through WSDL 2.0 extensions), this specification only defines a means of expressing constraints based upon the XML Infoset [[XML Information Set](#)]. Furthermore, although a number of alternate schema languages can be used to constrain the XML Infoset (as long as they support the semantics of either inlining or importing schema), this specification only defines the use of XML Schema [[XML Schema: Structures](#)], [[XML Schema: Datatypes](#)].

Specifically, the {[element declarations](#)} and {[type definitions](#)} properties of the [Description](#) component are collections of imported and inlined schema components that describe Infoset *element information items*.

When extensions are used to enable the use of a non-Infoset data model, or a non-Schema constraint language, the `wsdl:required` attribute information item MAY be used to require support for that extension.

Note:

Support for the W3C XML Schema [[XML Schema: Structures](#)], [[XML Schema: Datatypes](#)] is included in the conformance criteria for WSDL 2.0 documents (see [3.1 Using W3C XML Schema Definition Language](#)).

The schema components contained in the {[element declarations](#)} property of the [Description](#) component provide the type system used for [Interface Message Reference](#) and [Interface Fault](#) components. [Interface Message Reference](#) components indicate their structure and content by using the standard *attribute information items* element, or for alternate schema languages in which these concepts do not map well, by using alternative *attribute information item* extensions. [Interface Fault](#) components behave similarly. Such extensions should define how they reference type system components. Such type system components MAY appear in additional collection properties on the [Description](#) component.

Extensions in the form of *attribute information items* can be used to refer to constraints (type definitions or analogous constructs) described using other schema languages or type systems. Such components MAY appear in additional collection properties on the [Description](#) component.

The types *element information item* encloses data type definitions, based upon the XML Infoset, used to define messages and has the following Infoset properties:

- A [local name] of `types`.
- A [namespace name] of `"http://www.w3.org/ns/wsdl"`.
- Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT `http://www.w3.org/ns/wsdl`
- Zero or more *element information items* amongst its [children] as follows:
 - Zero or more documentation *element information items* (see [5. Documentation](#)) in its [children] property.
 - Zero or more *element information items* from among the following, in any order:
 - `xs:import` *element information items*
 - `xs:schema` *element information items*
 - Other namespace qualified *element information items* whose namespace is NOT `http://www.w3.org/ns/wsdl`

3.1 Using W3C XML Schema Definition Language

XML Schema MAY be used as the schema language via import or inlining.

A WSDL 2.0 document MUST NOT refer to XML Schema components in a given namespace UNLESS an `xs:import` or `xs:schema` *element information item* for that namespace is present OR the namespace is the XML Schema namespace, `http://www.w3.org/2001/XMLSchema`, which contains built-in types as defined in XML Schema Part 2: Datatypes Second Edition [[XML Schema: Datatypes](#)].[‡] That is, using the `xs:import` or `xs:schema` *element information item* is a necessary condition for making XML Schema components, other than the built-in components, referenceable within a WSDL 2.0 document. The built-in XML Schema datatypes are built-in to the WSDL 2.0 component model and are contained in the {[type definitions](#)} property of the [Description](#) component. A WSDL 2.0 document that refers to any element declaration or type definition component of the XML Schema namespace, except the built-in primitive and derived types, MUST import `http://www.w3.org/2001/XMLSchema`.

[Table 3-1](#) summarizes the referenceability of schema components.

Table 3-1. Referenceability of schema components

	XML Representation	Referenceability of XML Schema Components
Including description	<code>description/include</code>	XML Schema components in the included Description component's { element declarations } and { type definitions } properties are referenceable.
Importing description	<code>description/import</code>	None of the XML Schema Components in the imported Description component are referenceable.
Importing XML Schema	<code>description/types/xs:import</code>	Element Declaration and Type Definition components in the imported namespace are referenceable.

Inlined XML Schema	description/types/xs:schema	Element Declaration and Type Definition components in the inlined XML Schema are referenceable.
--------------------	-----------------------------	---

3.1.1 Importing XML Schema

Importing an XML Schema uses the syntax and semantics of the `xs:import` mechanism defined by XML Schema [[XML Schema: Structures](#)], [[XML Schema: Datatypes](#)], with the differences defined in this section and the following one. The schema components defined in the imported namespace are referenceable by QName (see [2.17 QName resolution](#)). Only components in the imported namespace are referenceable in the WSDL 2.0 document. For each component in the imported namespace, a corresponding [Element Declaration](#) component or [Type Definition](#) component MUST appear in the `{element declarations}` or `{type definitions}` property respectively of the [Description](#) component corresponding to the WSDL document that imports the schema, or that imports directly or indirectly a WSDL document that imports the schema.[‡] Schema components not in an imported namespace MUST NOT appear in the `{element declarations}` or `{type definitions}` properties.[‡]

A child *element information item* of the `types element information item` is defined with the Infoset properties as follows:

- A [local name] of "import".
- A [namespace name] of "http://www.w3.org/2001/XMLSchema".
- One or two *attribute information items* as follows:
 - A REQUIRED `namespace` *attribute information item* as described below.
 - An OPTIONAL `schemaLocation` *attribute information item* as described below.

3.1.1.1 namespace attribute information item

The `namespace attribute information item` defines the namespace of the element declarations and type definitions imported from the referenced schema. The referenced schema MUST contain a `targetNamespace attribute information item` on its `xs:schema element information item`.[‡] The value of the `targetNamespace attribute information item` of the `xs:schema element information item` of an imported schema MUST equal the value of the `namespace` of the `import element information item` in the importing WSDL 2.0 document.[‡] Note that a WSDL 2.0 document must not import a schema that does not have a `targetNamespace attribute information item` on its `xs:schema element information item`. Such schemas must first be included (using `xs:include`) in a schema that contains a `targetNamespace attribute information item` on its `xs:schema element information item`, which can then be either imported or inlined in the WSDL 2.0 document.

The `namespace attribute information item` has the following Infoset properties:

- A [local name] of `namespace`
- A [namespace name] which has no value.

The type of the `namespace attribute information item` is `xs:anyURI`.

3.1.1.2 schemaLocation attribute information item

The `schemaLocation attribute information item`, if present, provides a hint to the XML Schema processor as to where the schema may be located. Caching and cataloging technologies may provide better information than this hint. The `schemaLocation attribute information item` has the following Infoset properties:

- A [local name] of `schemaLocation`.
- A [namespace name] which has no value.

The type of the `schemaLocation attribute information item` is `xs:anyURI`.

Every QName reference must resolve (see [2.17 QName resolution](#)). Note that, when resolving QNames references for schema definitions, the namespace must be imported by the referring WSDL 2.0 document (see [3.1 Using W3C XML Schema Definition Language](#)).

3.1.2 Inlining XML Schema

Inlining an XML schema uses the existing top-level `xs:schema element information item` defined by XML Schema [[XML Schema: Structures](#)]. Conceptually, inlining can be viewed as simply cutting and pasting an existing schema document to a location inside the `types element information item`.

The schema components defined and declared in the inlined schema document are referenceable by QName (see [2.17 QName resolution](#)). Only components defined and declared in the schema itself and components included by it via `xs:include` are referenceable. For each component defined and declared in the inlined schema document or included by `xs:include`, a corresponding [Element Declaration](#) component or [Type Definition](#) component MUST appear in the `{element declarations}` property or `{type definitions}` property respectively of the [Description](#) component

corresponding to the WSDL document that contains the schema, or that imports directly or indirectly a WSDL document that contains the schema.[‡] Schema components not defined or declared in the inlined schema document or included by `xs:include` MUST NOT appear in the `{element declarations}` or `{type definitions}` properties.[‡]

Note that components in the namespace that the inline schema imports via `xs:import` are not automatically referenceable from the WSDL 2.0 document that contains the inline schema. If the namespace referenced in a QName is contained in an inline schema, it MAY be imported without a `schemaLocation` attribute, so long as the inline schema has been resolved in the current component model.

Note that components defined in an inlined XML schema are not automatically referenceable within the WSDL 2.0 document that imported (using `wsdl:import`) the WSDL 2.0 document that inlines the schema (see [4.2 Importing Descriptions](#) for more details). For this reason, it is recommended that XML schema documents intended to be shared across several WSDL 2.0 documents be placed in separate XML schema documents and imported using `xs:import`, rather than inlined inside a WSDL 2.0 document.

Inside an inlined XML schema, the `xs:import` and `xs:include` *element information items* MAY be used to refer to other XML schemas inlined in the same or other WSDL 2.0 document, provided that an appropriate value, such as a fragment identifier (see [\[XML Schema: Structures\]](#) 4.3.1) is specified for their `schemaLocation` *attribute information items*. For `xs:import`, the `schemaLocation` attribute is not required so long as the namespace has been resolved in the current component model. The semantics of such *element information items* are governed solely by the XML Schema specification [\[XML Schema: Structures\]](#).

A WSDL 2.0 document MAY inline two or more schemas from the same `targetNamespace`. For example, two or more inlined schemas can have the same `targetNamespace` provided that they do not define the same elements or types. A WSDL 2.0 document MUST NOT define the same element or type in more than one inlined schema.[‡] Note that it is the responsibility of the underlying XML Schema processor to sort out a coherent set of schema components.

The `xs:schema` *element information item* has the following Infoset properties:

- A [local name] of schema.
- A [namespace name] of "http://www.w3.org/2001/XMLSchema".
- Additional OPTIONAL *attribute information items* as specified for the `xs:schema` *element information item* by the XML Schema specification.
- Zero or more child *element information items* as specified for the `xs:schema` *element information item* by the XML Schema specification.

3.1.3 References to Element Declarations and Type Definitions

Whether inlined or imported, the global element declarations present in a schema are referenceable from an [Interface Message Reference](#) or [Interface Fault](#) component. Similarly, regardless of whether they are inlined or imported, the global type definitions present in a schema are referenceable from other components.

A named, global `xs:element` declaration is referenceable from the *element attribute information item* of an input, output (see [2.5.2 XML Representation of Interface Message Reference Component](#)) or fault *element information item* (see [2.3.2 XML Representation of Interface Fault Component](#)). The QName of the element declaration is constructed from the `targetNamespace` of the schema and the value of the *name attribute information item* of the `xs:element` *element information item*. Note that the *element attribute information item* cannot refer to a global `xs:simpleType` or `xs:complexType` definition, since these are in a different symbol space than global element declarations. If the *element attribute information item* erroneously contains the QName of a type definition then this would result in a failure to resolve the element declaration.

3.2 Using Other Schema Languages

Since it is unreasonable to expect that a single schema language can be used to describe all possible [Interface Message Reference](#) and [Interface Fault](#) component contents and their constraints, WSDL 2.0 allows alternate schema languages to be specified via extension elements. An extension *element information item* MAY appear under the *types element information item* to identify the schema language employed, and to locate the schema instance defining the grammar for [Interface Message Reference](#) and [Interface Fault](#) components. Depending upon the schema language used, an *element information item* MAY be defined to allow inlining, if and only if the schema language can be expressed in XML.

A specification of extension syntax for an alternative schema language MUST include the declaration of an *element information item*, intended to appear as a child of the `wsdl:types` *element information item*, which references, names, and locates the schema instance (an *import element information item*).[‡] The extension specification SHOULD, if necessary, define additional properties of the [Description](#) component (and extension attributes) to hold the components of the referenced type system. It is expected that additional extension attributes for [Interface Message Reference](#) and [Interface Fault](#) components will also be defined, along with a mechanism for resolving the values of those attributes to a particular imported type system component.

A specification of extension syntax for an alternative schema language MUST use a namespace that is different than the namespace of XML Schema.[‡] The namespace of the alternative schema language is used for *element information items* that are children of the `wsdl:types` *element information item* and for any extension *attribute information items* that appear on other components. The namespace used for an alternate schema language MUST be an absolute IRI.[‡]

See [\[WSDL 2.0 Alternative Schema Languages Support\]](#) for examples of using other schema languages. These examples reuse the `{element declarations}` property of the [Description](#) component and the *element attribute*

information items of the `wsdl:input`, `wsdl:output` and `wsdl:fault` *element information items*.

Note:

This specification does not define the behavior of a WSDL 2.0 document that uses multiple schema languages for describing type system components simultaneously.

3.3 Describing Messages that Refer to Services and Endpoints

Web services can exchange messages that refer to other Web services or Web service endpoints. If the interface or binding of these referenced services or endpoints are known at description time, then it may be useful to include this information in the WSDL 2.0 document that describes the Web service. WSDL 2.0 provides two global *attribute information items*, `wsdlx:interface` and `wsdlx:binding` that may be used to annotate XML Schema components or components from other type description languages.

WSDL 2.0 defines the use of these global *attribute information items* to annotate XML Schema components that use the `xs:anyURI` simple type in an *element information item* or *attribute information item* for endpoint addresses that correspond to the `{address}` property of the [Endpoint](#) component. However, the use of these global *attribute information items* is not limited to simple types based on `xs:anyURI`. They may be used for any other types that are used to refer to Web services or Web service endpoints, e.g. a WS-Addressing Endpoint Reference [[WSA 1.0 Core](#)]. See the primer [[WSDL 2.0 Primer](#)] for more information and examples.

3.3.1 `wsdlx:interface` *attribute information item*

WSDL 2.0 provides a global *attribute information item* with the following Infoset properties:

- A [local name] of `interface`.
- A [namespace name] of " `http://www.w3.org/ns/wsdl-extensions` ".

The type of the `wsdlx:interface` *attribute information item* is an `xs:QName` that specifies the `{name}` property of an [Interface](#) component.[‡]

3.3.2 `wsdlx:binding` *attribute information item*

WSDL 2.0 provides a global *attribute information item* with the following Infoset properties:

- A [local name] of `binding`.
- A [namespace name] of " `http://www.w3.org/ns/wsdl-extensions` ".

The type of the `wsdlx:binding` *attribute information item* is an `xs:QName` that specifies the `{name}` property of a [Binding](#) component.[‡]

3.3.3 `wsdlx:interface` and `wsdlx:binding` Consistency

The `wsdlx:interface` and `wsdlx:binding` attributes may be used either independently or together. If `wsdlx:interface` and `wsdlx:binding` are used together then they MUST satisfy the same consistency rules that apply to the `{interface}` property of a [Service](#) component and the `{binding}` property of a nested [Endpoint](#) component, that is either the binding refers the interface of the service or the binding refers to no interface.[‡]

3.3.4 Use of `wsdlx:interface` and `wsdlx:binding` with `xs:anyURI`

`wsdlx:interface` and `wsdlx:binding` are used to describe *element information items* and *attribute information items* whose type is `xs:anyURI` or a restriction of it, as well messages that contain the `{address}` property of an [Endpoint](#). This is accomplished by including the `wsdlx:interface` and/or `wsdlx:binding` *attribute information item* in the `xs:element`, `xs:simpleType`, or `xs:attribute` *element information item* of the corresponding XML Schema component.

4. Modularizing WSDL 2.0 descriptions

WSDL 2.0 provides two mechanisms for modularizing WSDL 2.0 descriptions. These mechanisms help to make Web service descriptions clearer by allowing separation of the various components of a description. Such separation can be performed according to the level of abstraction of a given set of components, or according to the namespace affiliation required of a given set of components or even according to some other grouping such as application applicability.

Both mechanisms work at the level of WSDL 2.0 components and NOT at the level of XML Information Sets or XML 1.0 serializations.

4.1 Including Descriptions

```
<description>
  <include
    location="xs:anyURI" >
    <documentation />*
```

```

</include>
</description>

```

The WSDL 2.0 *include element information item* allows separating the different components of a service definition, belonging to the same target namespace, into independent WSDL 2.0 documents.

The WSDL 2.0 *include element information item* is modeled after the XML Schema *include element information item* (see [\[XML Schema: Structures\]](#), section 4.2.3 "References to schema components in the same namespace"). Specifically, it can be used to include components from WSDL 2.0 descriptions that share a target namespace with the including description. Components in the transitive closure of the included WSDL 2.0 documents become part of the [Description](#) component of the including WSDL 2.0 document. The included components can be referenced by QName. Note that because all WSDL 2.0 descriptions have a target namespace, no-namespace includes (sometimes known as "chameleon includes") never occur in WSDL 2.0.

A mutual include is the direct inclusion by one WSDL 2.0 document of another WSDL 2.0 document which includes the first document. A circular include achieves the same effect with greater indirection (for example, A includes B, B includes C, and C includes A). Multiple inclusion of a single WSDL 2.0 document resolves to a single set of components, as if the document was included only once. Mutual, multiple, and circular includes are explicitly permitted, and do not represent multiple redefinitions of the same components.

The *include element information item* has:

- A [local name] of *include*.
- A [namespace name] of "http://www.w3.org/ns/wsdl".
- One or more *attribute information items* amongst its [attributes] as follows:
 - A REQUIRED *location attribute information item* as described below in [4.1.1 location attribute information item with include \[owner element\]](#).
 - Zero or more namespace qualified *attribute information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".
- Zero or more *element information item* amongst its [children], as follows:
 - Zero or more *documentation element information items* (see [5. Documentation](#)).
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wsdl".

4.1.1 location *attribute information item* with include [owner element]

The *location attribute information item* has the following Infoset properties:

- A [local name] of *location*.
- A [namespace name] which has no value.

A *location attribute information item* is of type `xs:anyURI`. Its actual value is the location of some information about the namespace identified by the *targetNamespace attribute information item* of the containing description *element information item*.

The IRI indicated by *location* MUST resolve to a WSDL 2.0 document.[‡] (see [7. Locating WSDL 2.0 Documents](#))

The actual value of the *targetNamespace attribute information item* of the included WSDL 2.0 document MUST match the actual value of the *targetNamespace attribute information item* of the description *element information item* which is the [parent] of the *include element information item*.[‡]

4.2 Importing Descriptions

```

<description>
  <import
    namespace="xs:anyURI" location="xs:anyURI"? >
    <documentation />*
  </import>
</description>

```

Every top-level WSDL 2.0 component is associated with a namespace. Every WSDL 2.0 document carries a *targetNamespace attribute information item* on its `wsdl:description element information item`. This attribute associates the document with a target namespace, which consequently also becomes the namespace of each top-level WSDL 2.0 component defined in that document. Any namespace other than the document's target namespace is referred to as a *foreign namespace*. Any component associated with a foreign namespace is referred to as a *foreign component*. This section describes the syntax and mechanisms by which references may be made from within a WSDL 2.0 document to foreign components. In addition to this syntax, there is an optional facility for suggesting the IRI of a WSDL 2.0 document that contains definitions of foreign components.

The WSDL 2.0 *import element information item* is modeled after the XML Schema *import element information item* (see [\[XML Schema: Structures\]](#), section 4.2.3 "References to schema components across namespaces"). Specifically, it can be used to import WSDL 2.0 components from a foreign namespace. The WSDL 2.0 *import element information item* identifies a foreign namespace. The presence of a WSDL 2.0 *import element information item* signals that the WSDL 2.0 document may contain references to foreign components. The `wsdl:import element information item` is therefore like a forward declaration for foreign namespaces.

As with XML schema, any WSDL 2.0 document that references a foreign component **MUST** have a `wsdl:import` *element information item* for the associated foreign namespace (but which does not necessarily provide a `location` *attribute information item* that identifies the WSDL 2.0 document in which the referenced component is defined).[‡] In other respects, the visibility of components is pervasive: if two WSDL 2.0 documents import the same namespace, then they will have access to the same components from the imported namespace (i.e. regardless of which, if any, `location` *attribute information item* values are provided on the respective `wsdl:import` *element information items*.)

Using the `wsdl:import` *element information item* is a necessary condition for making foreign components available to a WSDL 2.0 document. That is, a WSDL 2.0 document can only refer to foreign components, if it contains an `wsdl:import` *element information item* for the associated foreign namespace.

If a WSDL 2.0 document contains more than one `wsdl:import` *element information item* for a given value of the `namespace` *attribute information item*, then they **MUST** provide different values for the `location` *attribute information item*.[‡] Repeating the `wsdl:import` *element information item* for the same `namespace` value **MAY** be used as a way to provide alternate locations to find information about a given namespace.

Furthermore, this specification **DOES NOT** require the `location` *attribute information item* to be dereferencable. When it is not dereferencable, no information about the imported namespace is provided by that `wsdl:import` *element information item*. It is possible that such lack of information can cause QNames in other parts of a WSDL 2.0 [Description](#) component to become broken references (see [2.17 QName resolution](#)). Such broken references are not ascribed to the `wsdl:import` *element information item*, but rather are failures of the QName resolution requirements which must be detected as described in [2.17 QName resolution](#).

The `import` *element information item* has the following Infoset properties:

- A [local name] of `import`.
- A [namespace name] of `"http://www.w3.org/ns/wsdl"`.
- One or more *attribute information items* amongst its [attributes] as follows:
 - A **REQUIRED** `namespace` *attribute information item* as described below in [4.2.1 namespace attribute information item](#).
 - An **OPTIONAL** `location` *attribute information item* as described below in [4.2.2 location attribute information item with import \[owner element\]](#).
 - Zero or more namespace-qualified *attribute information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"`.
- Zero or more *element information items* amongst its [children], as follows:
 - Zero or more *documentation element information items* (see [5. Documentation](#)).
 - Zero or more namespace-qualified *element information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"`.

4.2.1 namespace *attribute information item*

The `namespace` *attribute information item* has the following Infoset properties:

- A [local name] of `namespace`.
- A [namespace name] which has no value.

The `namespace` *attribute information item* is of type `xs:anyURI`. Its actual value indicates that the containing WSDL 2.0 document **MAY** contain qualified references to WSDL 2.0 components in that namespace (via one or more prefixes declared with namespace declarations in the normal way). This value **MUST NOT** match the actual value of `targetNamespace` *attribute information item* in the enclosing WSDL 2.0 document.[‡] If the `location` *attribute* in the `import` *element information item* is dereferencable, then it **MUST** reference a WSDL 2.0 document.[‡] If the `location` *attribute information item* of the `import` *element information item* is dereferencable, then the actual value of the `namespace` *attribute information item* **MUST** be identical to the actual value of the `targetNamespace` *attribute information item* of the referenced WSDL 2.0 document (see [7. Locating WSDL 2.0 Documents](#)).[‡]

4.2.2 location *attribute information item* with `import` [owner element]

The `location` *attribute information item* has the following Infoset properties:

- A [local name] of `location`.
- A [namespace name] which has no value.

The `location` *attribute information item* is of type `xs:anyURI`. Its actual value, if present, gives a hint as to where a serialization of a WSDL 2.0 document with definitions for the imported namespace can be found.

The `location` *attribute information item* is optional. This allows WSDL 2.0 components to be constructed from information other than an XML 1.0 serialization of a WSDL 2.0 document. It also allows the development of WSDL 2.0 processors that have *a priori* (i.e., built-in) knowledge of certain namespaces.

4.3 Extensions

The semantics of an extension **MUST NOT** depend on how components are brought into a component model instance via `<import>` or `<include>`.[‡] That is, the components that are defined by a WSDL 2.0 document are

determined by the contents of the document, EXCEPT for the resolution of references to other components that may be defined in other documents, AND any further processing, as mandated by the extension specification, that depends on such references having been resolved to the actual components.

This restriction on the behavior of extensions permits WSDL 2.0 documents to be flexibly modularized and efficiently processed. In contrast, note that the so-called chameleon include mechanism of XML Schema, which allows a no-namespace schema to be included in a schema document that has a namespace, violates this restriction since the namespace of the included XML Schema components is determined by the including XML Schema document (see 4.2.1 Assembling a schema for a single target namespace from multiple schema definition documents in [\[XML Schema: Structures\]](#)).

5. Documentation

```
<documentation>
  [extension elements]*
</documentation>
```

WSDL 2.0 uses the optional *documentation element information item* as a container for human readable or machine processable documentation. The content of the *element information item* is arbitrary *character information items* and *element information items* ("mixed" content in XML Schema [\[XML Schema: Structures\]](#)). The *documentation element information item* is allowed inside any WSDL 2.0 *element information item*.

Like other *element information items* in the "http://www.w3.org/ns/wSDL" namespace, the *documentation element information item* allows qualified *attribute information items* whose [namespace name] is not "http://www.w3.org/ns/wSDL". The `xml:lang` attribute (see [\[XML 1.0\]](#)) MAY be used to indicate the language used in the contents of the *documentation element information item*.

The *documentation element information item* has:

- A [local name] of *documentation*.
- A [namespace name] of "http://www.w3.org/ns/wSDL".
- Zero or more *attribute information items* in its [attributes] property.
- Zero or more child *element information items* in its [children] property.
- Zero or more *character information items* in its [children] property.

6. Language Extensibility

The schema for WSDL 2.0 has a two-part extensibility model based on namespace-qualified elements and attributes. An extension is identified by the QName consisting of its namespace IRI and its element or attribute name. The meaning of an extension SHOULD be defined (directly or indirectly) in a document that is available at its namespace IRI.[‡]

6.1 Element-based Extensibility

WSDL 2.0 allows extensions to be defined in terms of *element information items*. Where indicated herein, WSDL 2.0 allows namespace-qualified *element information items* whose [namespace name] is NOT "http://www.w3.org/ns/wSDL" to appear among the [children] of specific *element information items* whose [namespace name] is "http://www.w3.org/ns/wSDL". Such *element information items* MAY be used to annotate WSDL 2.0 constructs such as interface, operation, etc.

It is expected that extensions will add to the existing properties of components in the component model. The specification for an extension *element information item* should include definitions of any such properties and the mapping from the XML representation of the extension to the properties in the component model.

The WSDL 2.0 schema defines a base type for use by extension elements. [Example 6-1](#) shows the type definition. The use of this type as a base type is optional.

Example 6-1. Base type for extension elements

```
<xs:complexType name='ExtensionElement' abstract='true' >
  <xs:attribute ref='wSDL:required' use='optional' />
</xs:complexType>
```

Extension elements are commonly used to specify some technology-specific binding. They allow innovation in the area of network and message protocols without having to revise the base WSDL 2.0 specification. WSDL 2.0 recommends that specifications defining such protocols also define any necessary WSDL 2.0 extensions used to describe those protocols or formats.

6.1.1 Mandatory extensions

Extension elements can be marked as mandatory by annotating them with a `wSDL:required` *attribute information item* (see [6.1.2 required attribute information item](#)) with a value of "true". A mandatory extension is an extension that MAY change the meaning of the element to which it is attached, such that the meaning of that element is no longer governed by this specification. Instead, the meaning of an element containing a mandatory extension is governed by

the meaning of that extension. Thus, the definition of the element's meaning is *delegated* to the specification that defines the extension.

An extension that is NOT marked as mandatory MUST NOT invalidate the meaning of any part of a WSDL 2.0 document.[‡] Thus, a NON-mandatory extension merely provides additional description of capabilities of the service. This specification does not provide a mechanism to mark extension attributes as being required. Therefore, all extension attributes are NON-mandatory.

Note:

A mandatory extension is considered mandatory because it has the ability to change the meaning of the element to which it is attached. Thus, the meaning of the element may not be fully understood without understanding the attached extension. A NON-mandatory extension, on the other hand, can be safely ignored without danger of misunderstanding the rest of the WSDL 2.0 document.

If a WSDL 2.0 document declares an extension as optional (i.e., NON-mandatory), then the Web service MUST NOT assume that the client supports that extension *unless* the Web service knows (through some other means) that the client has in fact elected to engage and support that extension.[‡]

Note:

A key purpose of an extension is to formally indicate (i.e., in a machine-processable way) that a particular feature or convention is supported or required. This enables toolkits that understand the extension to engage it automatically, while toolkits that do not yet understand a required extension can possibly bring it to the attention of an operator for manual support.

If a Web service requires a client to follow a particular convention that is likely to be automatable in WSDL 2.0 toolkits, then that convention SHOULD be indicated in the WSDL 2.0 document as a `wsdl:required` extension, rather than just being conveyed out of band, even if that convention is not currently implemented in WSDL 2.0 toolkits.

This practice will help prevent interoperability problems that could arise if one toolkit requires a particular convention that is not indicated in the WSDL 2.0 document, while another toolkit does not realize that that convention is required. It will also help facilitate future automatic processing by WSDL 2.0 toolkits.

On the other hand, a client MAY engage an extension that is declared as optional in the WSDL 2.0 document. Therefore, the Web service MUST support every extension that is declared as optional in the WSDL 2.0 document, in addition to supporting every extension that is declared as mandatory.[‡]

Note:

If finer-grain, direction-sensitive control of extensions is desired, then such extensions may be designed in a direction-sensitive manner (from the client or from the Web service) so that either direction may be separately marked required or optional. For example, instead of defining a single extension that governs both directions, two extensions could be defined -one for each direction.

Validity of a WSDL 2.0 document can only be assessed within the context of a set of supported extensions. A WSDL 2.0 document that contains a required but unsupported extension is invalid with respect to that set of supported extensions.

6.1.2 required *attribute information item*

WSDL 2.0 provides a global *attribute information item* with the following Infoset properties:

- A [local name] of `required`.
- A [namespace name] of `"http://www.w3.org/ns/wsdl"`.

The type of the `required attribute information item` is `xs:boolean`. Its default value is `"false"` (hence extensions are NOT required by default).

6.2 Attribute-based Extensibility

WSDL 2.0 allows qualified *attribute information items* whose [namespace name] is NOT `"http://www.w3.org/ns/wsdl"` to appear on any *element information item* whose namespace name IS `"http://www.w3.org/ns/wsdl"`. Such *attribute information items* can be used to annotate WSDL 2.0 constructs such as interfaces, bindings, etc.

WSDL 2.0 does not provide a mechanism for marking extension *attribute information items* as mandatory.

6.3 Extensibility Semantics

As indicated above, it is expected that the presence of extension elements and attributes will result in additional properties appearing in the component model.

The presence of an optional extension element or attribute MAY therefore augment the semantics of a WSDL 2.0 document in ways that do not invalidate the existing semantics. However, the presence of a mandatory extension element MAY alter the semantics of a WSDL 2.0 document in ways that invalidate the existing semantics.

Extension elements SHOULD NOT alter the existing semantics in ways that are likely to confuse users.

Note:

Note that, however, once the client and service both know that an optional extension has been engaged (because the service has received a message explicitly engaging that extension, for example), then the semantics of that extension supersede what the WSDL 2.0 document indicated. For example, the WSDL 2.0

document may have specified an XML message schema to be used, but also indicated an optional security extension that encrypts the messages. If the security extension is engaged, then the encrypted messages will no longer conform to the specified message schema (until they are decrypted).

Note:

Authors of extension elements should make sure to include in the specification of these elements a clear statement of the requirements for document conformance (see [1.3 Document Conformance](#)).

Note:

Authors of extension elements that may manifest as properties of the [Description](#) component should be aware of the impact of imports on their extensions, or of their extensions on imports. It is not possible, within the component model, to define extensions that have an effective scope equal to the scope of a containing file. Extensions that modify the behavior of the components contained in a description may therefore unexpectedly modify the behavior of components in imported descriptions as well, unless proper care is taken.

7. Locating WSDL 2.0 Documents

A *WSDL 2.0 document* is a *description element information item* that is either the document root of an XML document or an element within an XML document. The *location* of a WSDL 2.0 MAY therefore be specified by an IRI for an XML resource whose document root is a *description element information item* or an IRI-reference for a *description element information item* within an XML resource.

As an XML vocabulary, WSDL 2.0 documents, WSDL2.0 document fragments or QName references to WSDL 2.0 components MAY appear within other XML documents. This specification defines a global attribute, `wsdlLocation`, to help with QName resolution (see [2.17 QName resolution](#)). This attribute allows an element that contains such references to be annotated to indicate where the WSDL 2.0 documents for one or more namespaces can be found. In particular, this attribute is expected to be useful when using service references in message exchanges.

The `wsdlLocation` global attribute is defined in the namespace "http://www.w3.org/ns/wsdl-instance" (hereafter referred to as "wsdl:wsdlLocation", for brevity). This attribute MAY appear on any XML element which allows attributes from other namespaces to occur. It MUST NOT appear on a `wsdl:description` element or any of its children/descendants.[‡]

A normative XML Schema [[XML Schema: Structures](#)], [[XML Schema: Datatypes](#)] document for the "http://www.w3.org/ns/wsdl-instance" namespace can be found at <http://www.w3.org/ns/wsdl-instance>.

7.1 wsdl:wsdlLocation attribute information item

WSDL 2.0 provides a global *attribute information item* with the following Infoset properties:

- A [local name] of `wsdlLocation`.
- A [namespace name] of "http://www.w3.org/ns/wsdl-instance".

The type of the `wsdlLocation` *attribute information item* is a list `xs:anyURI`. Its actual value MUST be a list of pairs of IRIs; where the first IRI of a pair, which MUST be an absolute IRI as defined in [[IETF RFC 3987](#)], indicates a WSDL 2.0 (or 1.1) namespace name, and, the second a hint as to the location of a WSDL 2.0 document defining WSDL 2.0 components (or WSDL 1.1 elements [[WSDL 1.1](#)]) for that namespace name.[‡] The second IRI of a pair MAY be absolute or relative. For each pair of IRIs, if the location IRI of the pair is dereferencable, then it MUST reference a WSDL 2.0 (or 1.1) document whose target namespace is the namespace IRI of the pair.[‡]

8. Conformance

This section describes how this specification conforms to other specifications. This is limited, at present, to the XML Information Set specification. Refer to [1.3 Document Conformance](#) for a description of the criteria that Web service description documents must satisfy in order to conform to this specification.

8.1 XML Information Set Conformance

This specification conforms to the [[XML Information Set](#)]. The following information items MUST be present in the input Infosets to enable correct processing of WSDL 2.0 documents:

- *Document Information Items* with [children] and [base URI] properties.
- *Element Information Items* with [namespace name], [local name], [children], [attributes], [base URI] and [parent] properties.
- *Attribute Information Items* with [namespace name], [local name] and [normalized value] properties.
- *Character Information Items* with [character code], [element content whitespace] and [parent] properties.

9. XML Syntax Summary (Non-Normative)

```
<description targetNamespace="xs:anyURI" >
  <documentation />*

  <import namespace="xs:anyURI" location="xs:anyURI"? >
```

```

    <documentation />*
</import>*

<include location="xs:anyURI" >
    <documentation />*
</include>*

<types>
    <documentation />*

    [ <xs:import namespace="xs:anyURI" schemaLocation="xs:anyURI"? /> |
      <xs:schema targetNamespace="xs:anyURI"? /> |
        other extension elements ]*
</types>

<interface name="xs:NCName" extends="List of xs:QName"? styleDefault="List of xs:anyURI"? >
    <documentation />*

    <fault name="xs:NCName" element="union of xs:QName, xs:token"? >
        <documentation />*
    </fault>*

    <operation name="xs:NCName" pattern="xs:anyURI"? style="List of xs:anyURI"? >
        <documentation />*

        <input messageLabel="xs:NCName"? element="union of xs:QName, xs:token"? >
            <documentation />*
        </input>*

        <output messageLabel="xs:NCName"? element="union of xs:QName, xs:token"? >
            <documentation />*

        </output>*

        <infault ref="xs:QName" messageLabel="xs:NCName"? >
            <documentation />*
        </infault>*

        <outfault ref="xs:QName" messageLabel="xs:NCName"? >
            <documentation />*
        </outfault>*

    </operation>*
</interface>*

<binding name="xs:NCName" interface="xs:QName"? type="xs:anyURI" >
    <documentation />*

    <fault ref="xs:QName" >
        <documentation />*
    </fault>*

    <operation ref="xs:QName" >
        <documentation />*

        <input messageLabel="xs:NCName"? >
            <documentation />*
        </input>*

        <output messageLabel="xs:NCName"? >
            <documentation />*
        </output>*

        <infault ref="xs:QName" messageLabel="xs:NCName"? >
            <documentation />*
        </infault>*

        <outfault ref="xs:QName" messageLabel="xs:NCName"? >
            <documentation />*
        </outfault>*

    </operation>*
</binding>*

<service name="xs:NCName" interface="xs:QName" >
    <documentation />*

    <endpoint name="xs:NCName" binding="xs:QName" address="xs:anyURI"? >
        <documentation />*
    </endpoint>+

</service>*
</description>

```

10. References

10.1 Normative References

[Character Model for the WWW]

[Character Model for the World Wide Web 1.0: Fundamentals](#), M. Dürst, F. Yergeau, R. Ishida, M. Wolf, T. Texin, Editors. W3C Recommendation, 15 February 2005. Latest version available at <http://www.w3.org/TR/charmod/>.

[IETF RFC 2119]

[Key words for use in RFCs to Indicate Requirement Levels](#), S. Bradner, Author. Internet Engineering Task Force, March 1997. Available at <http://www.ietf.org/rfc/rfc2119.txt>.

[IETF RFC 3023]

[XML Media Types](#), M. Murata, S. St. Laurent, D. Kohn, Authors. Internet Engineering Task Force, January 2001. Available at <http://www.ietf.org/rfc/rfc3023.txt>.

[IETF RFC 3986]

[Uniform Resource Identifiers \(URI\): Generic Syntax](#), T. Berners-Lee, R. Fielding, L. Masinter, Authors. Internet Engineering Task Force, January 2005. Available at <http://www.ietf.org/rfc/rfc3986.txt>.

[IETF RFC 3987]

[Internationalized Resource Identifiers \(IRIs\)](#), M. Duerst, M. Suignard, Authors. Internet Engineering Task Force, January 2005. Available at <http://www.ietf.org/rfc/rfc3987.txt>.

[XLink 1.0]

[XML Linking Language \(XLink\) Version 1.0](#), Steve DeRose, Eve Maler, David Orchard, Editors. World Wide Web Consortium, 27 June 2001. This version of the XLink Recommendation is <http://www.w3.org/TR/2001/REC-xlink-20010627/>. The [latest version of XLink](#) is available at <http://www.w3.org/TR/xlink/>.

[XML 1.0]

[Extensible Markup Language \(XML\) 1.0 \(Fourth Edition\)](#), T. Bray, J. Paoli, C. M. Sperberg-McQueen, E. Maler, and F. Yergeau, Editors. World Wide Web Consortium, 10 February 1998, revised 16 August 2006. This version of the XML 1.0 Recommendation is <http://www.w3.org/TR/2006/REC-xml-20060816/>. The [latest version of "Extensible Markup Language \(XML\) 1.0"](#) is available at <http://www.w3.org/TR/REC-xml>.

[XML Namespaces]

[Namespaces in XML \(Second Edition\)](#), T. Bray, D. Hollander, A. Layman, and R. Tobin, Editors. World Wide Web Consortium, 16 August 2006. This version of the XML Information Set Recommendation is <http://www.w3.org/TR/2006/REC-xml-names-20060816>. The [latest version of Namespaces in XML](#) is available at <http://www.w3.org/TR/REC-xml-names>.

[XML Information Set]

[XML Information Set \(Second Edition\)](#), J. Cowan and R. Tobin, Editors. World Wide Web Consortium, 24 October 2001, revised 4 February 2004. This version of the XML Information Set Recommendation is <http://www.w3.org/TR/2004/REC-xml-infoset-20040204>. The [latest version of XML Information Set](#) is available at <http://www.w3.org/TR/xml-infoset>.

[XML Schema: Structures]

[XML Schema Part 1: Structures Second Edition](#), H. Thompson, D. Beech, M. Maloney, and N. Mendelsohn, Editors. World Wide Web Consortium, 2 May 2001, revised 28 October 2004. This version of the XML Schema Part 1 Recommendation is <http://www.w3.org/TR/2004/REC-xmlschema-1-20041028>. The [latest version of XML Schema Part 1](#) is available at <http://www.w3.org/TR/xmlschema-1>.

[XML Schema: Datatypes]

[XML Schema Part 2: Datatypes Second Edition](#), P. Byron and A. Malhotra, Editors. World Wide Web Consortium, 2 May 2001, revised 28 October 2004. This version of the XML Schema Part 2 Recommendation is <http://www.w3.org/TR/2004/REC-xmlschema-2-20041028>. The [latest version of XML Schema Part 2](#) is available at <http://www.w3.org/TR/xmlschema-2>.

[WSDL 2.0 Adjuncts]

[Web Services Description Language \(WSDL\) Version 2.0 Part 2: Adjuncts](#), R. Chinnici, H. Haas, A. Lewis, J.-J. Moreau, D. Orchard, S. Weerawarana, Editors. World Wide Web Consortium, 26 June 2007. This version of the "Web Services Description Language (WSDL) Version 2.0 Part 2: Adjuncts" Recommendation is available at <http://www.w3.org/TR/2007/REC-wsdl20-adjuncts-20070626>. The [latest version of "Web Services Description Language \(WSDL\) Version 2.0 Part 2: Adjuncts"](#) is available at <http://www.w3.org/TR/wsdl20-adjuncts>.

10.2 Informative References

[IETF RFC 2045]

[Multipurpose Internet Mail Extensions \(MIME\) Part One: Format of Internet Message Bodies](#), N. Freed, N. Borenstein, Authors. Internet Engineering Task Force, November 1996. Available at <http://www.ietf.org/rfc/rfc2045.txt>.

[IETF RFC 2616]

[Hypertext Transfer Protocol -- HTTP/1.1](#), R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, T. Berners-Lee, Authors. Internet Engineering Task Force, June 1999. Available at <http://www.ietf.org/rfc/rfc2616.txt>.

[WSA 1.0 Core]

[Web Services Addressing 1.0 - Core](#), M. Gudgin, M. Hadley, T. Rogers, Editors. World Wide Web Consortium, 9 May 2006. This version of Web Services Addressing 1.0 - Core Recommendation is <http://www.w3.org/TR/2006/REC-ws-addr-core-20060509/>. The latest version of the "Web Services Addressing 1.0 - Core" document is available from <http://www.w3.org/TR/ws-addr-core>.

[WSDL 1.1]

[Web Services Description Language \(WSDL\) 1.1](#), E. Christensen, F. Curbera, G. Meredith, and S. Weerawarana, Authors. World Wide Web Consortium, 15 March 2002. This version of the Web Services

Description Language 1.1 Note is <http://www.w3.org/TR/2001/NOTE-wsdl-20010315>. The [latest version of Web Services Description Language 1.1](#) is available at <http://www.w3.org/TR/wsdl>.

[WSDL 2.0 Primer]

[Web Services Description Language \(WSDL\) Version 2.0 Part 0: Primer](#), D.Booth, C.K. Liu, Editors. World Wide Web Consortium, 26 June 2007. This version of the "Web Services Description Language (WSDL) Version 2.0 Part 0: Primer" Recommendation is available at <http://www.w3.org/TR/2007/REC-wsdl20-primer-20070626>. The [latest version of "Web Services Description Language \(WSDL\) Version 2.0 Part 0: Primer"](#) is available at <http://www.w3.org/TR/wsdl20-primer>.

[WSDL 2.0 Requirements]

[Web Services Description Requirements](#), J. Schlimmer, Editor. World Wide Web Consortium, 28 October 2002. This version of the Web Services Description Requirements document is <http://www.w3.org/TR/2002/WD-ws-desc-reqs-20021028>. The [latest version of Web Services Description Requirements](#) is available at <http://www.w3.org/TR/ws-desc-reqs>.

[WSDL 2.0 Alternative Schema Languages Support]

[Discussion of Alternative Schema Languages and Type System Support in WSDL 2.0](#), A. Lewis, B. Parsia, Editors. World Wide Web Consortium, 17 August 2005. This version of the "Discussion of Alternative Schema Languages and Type System Support in WSDL 2.0" Working Group Note is <http://www.w3.org/TR/2005/NOTE-wsdl20-altschemalangs-20050817/>. The [latest version of "Discussion of Alternative Schema Languages and Type System Support in WSDL 2.0"](#) is available at <http://www.w3.org/TR/wsdl20-altschemalangs>.

[SOAP 1.2 Part 1: Messaging Framework (Second Edition)]

[SOAP Version 1.2 Part 1: Messaging Framework \(Second Edition\)](#), M. Gudgin, et al., Editors. World Wide Web Consortium, 24 June 2003, revised 27 April 2007. This version of the "SOAP Version 1.2 Part 1: Messaging Framework (Second Edition)" Recommendation is <http://www.w3.org/TR/2007/REC-soap12-part1-20070427/>. The [latest version of "SOAP Version 1.2 Part 1: Messaging Framework"](#) is available at <http://www.w3.org/TR/soap12-part1/>.

[XPointer]

[XPointer Framework](#), P. Grosso, E. Maler, J. Marsh, N. Walsh, Editors. World Wide Web Consortium, 25 March 2003. This version of the XPointer Framework Recommendation is <http://www.w3.org/TR/2003/REC-xptr-framework-20030325/>. The [latest version of XPointer Framework](#) is available at <http://www.w3.org/TR/xptr-framework/>.

[Z Notation Reference Manual]

[The Z Notation: A Reference Manual, Second Edition](#), J. M. Spivey, Prentice Hall, 1992.

[Fuzz 2000]

[Release Notes For Fuzz 2000](#), J. M. Spivey.

A. The application/wsdl+xml Media Type

This appendix defines the "application/wsdl+xml" media type which can be used to describe WSDL 2.0 documents serialized as XML.

A.1 Registration

MIME media type name:

application

MIME subtype name:

wsdl+xml

Required parameters:

none

Optional parameters:

charset

This parameter has identical semantics to the charset parameter of the "application/xml" media type as specified in [\[IETF RFC 3023\]](#).

Encoding considerations:

Identical to those of "application/xml" as described in [\[IETF RFC 3023\]](#), section 3.2, as applied to the WSDL document Infoset.

Security considerations:

See section [A.3 Security considerations](#).

Interoperability considerations:

There are no known interoperability issues.

Published specifications:

This document and [\[WSDL 2.0 Adjuncts\]](#).

Applications which use this media type:

No known applications currently use this media type.

Additional information:

File extension:

wsdl

Fragment identifiers:

Either a syntax identical to that of "application/xml" as described in [\[IETF RFC 3023\]](#), section 5 or the syntax defined in [A.2 Fragment Identifiers](#).

Base URI:

As specified in [\[IETF RFC 3023\]](#), section 6.

Macintosh File Type code:

WSDL

Person and email address to contact for further information:

World Wide Web Consortium <web-human@w3.org>

Intended usage:

COMMON

Author/Change controller:

The WSDL 2.0 specification set is a work product of the World Wide Web Consortium's [Web Service Description Working Group](#). The W3C has change control over these specifications.

A.2 Fragment Identifiers

This section defines a fragment identifier syntax for identifying components of a WSDL 2.0 document. This fragment identifier syntax is compliant with the [XPointer](#).

A WSDL 2.0 fragment identifier is an XPointer [\[XPointer\]](#), augmented with WSDL 2.0 pointer parts as defined below. Note that many of these parts require the pre-appearance of one or more `xmlns` pointer parts (see [3.4 Namespace Binding Context](#) in [XPointer](#)). The pointer parts have a scheme name that corresponds to one of the standard WSDL 2.0 component types, and scheme data that is a path composed of names that identify the components. The scheme names all begin with the prefix "wsdl." to avoid name conflicts with other schemes. The names in the path are of type either QName, NCName, IRI, URI, or Pointer Part depending on the context. The scheme data for WSDL 2.0 extension components is defined by the corresponding extension specification.

For QNames, any prefix MUST be defined by a preceding `xmlns` pointer part.[†] If a QName does not have a prefix then its namespace name is the target namespace of the WSDL 2.0 document.

The fragment identifier is typically constructed from the `{name}` property of the component and the `{name}` properties of its ancestors as a path according to [Table A-1](#). The first column of this table gives the name of the WSDL 2.0 component. Columns labeled 1 through 4 specify the identifiers that uniquely identify the component within its context. Identifiers are typically formed from the `{name}` property, although in several cases references to other components are used. These identifiers are then used to construct the pointer part in the last column. The fragment identifier in a WSDL 2.0 component IRI-reference MUST resolve to some component as defined by the construction rules in [Table A-1](#).[†]

Table A-1. Rules for determining pointer parts for WSDL 2.0 components

Component	1	2	3	4	Pointer Part
Description	n/a	n/a	n/a	n/a	wsdl.description()
Element Declaration	<i>element</i> QName	n/a	n/a	n/a	wsdl.elementDeclaration(element)
Element Declaration	<i>element</i> QName	<i>system</i> IRI	n/a	n/a	wsdl.elementDeclaration(element,system)
Type Definition	<i>type</i> QName	n/a	n/a	n/a	wsdl.typeDefinition(type)
Type Definition	<i>type</i> QName	<i>system</i> IRI	n/a	n/a	wsdl.typeDefinition(type,system)
Interface	<i>interface</i> NCName	n/a	n/a	n/a	wsdl.interface(interface)
Interface Fault	<i>interface</i> NCName	<i>fault</i> NCName	n/a	n/a	wsdl.interfaceFault(interface/fault)
Interface Operation	<i>interface</i> NCName	<i>operation</i> NCName	n/a	n/a	wsdl.interfaceOperation(interface/operation)
Interface Message Reference	<i>interface</i> NCName	<i>operation</i> NCName	<i>message</i> NCName	n/a	wsdl.interfaceMessageReference(interface/operation/message)
Interface Fault Reference	<i>interface</i> NCName	<i>operation</i> NCName	<i>message</i> NCName	<i>fault</i> QName	wsdl.interfaceFaultReference(interface/operation/message/fault)
Binding	<i>binding</i> NCName	n/a	n/a	n/a	wsdl.binding(binding)
Binding Fault	<i>binding</i> NCName	<i>fault</i> QName	n/a	n/a	wsdl.bindingFault(binding/fault)

Binding Operation	<i>binding</i> NCName	<i>operation</i> QName	n/a	n/a	wsdl.bindingOperation (<i>binding/operation</i>)
Binding Message Reference	<i>binding</i> NCName	<i>operation</i> QName	<i>message</i> NCName	n/a	wsdl.bindingMessageReference (<i>binding/operation/message</i>)
Binding Fault Reference	<i>binding</i> NCName	<i>operation</i> QName	<i>message</i> NCName	<i>fault</i> QName	wsdl.bindingFaultReference (<i>binding/operation/message/fault</i>)
Service	<i>service</i> NCName	n/a	n/a	n/a	wsdl.service (<i>service</i>)
Endpoint	<i>service</i> NCName	<i>endpoint</i> NCName	n/a	n/a	wsdl.endpoint (<i>service/endpoint</i>)
Extensions	<i>namespace</i> URI	<i>identifier</i> extension- specific- syntax	n/a	n/a	wsdl.extension (<i>namespace,identifier</i>)

Note that the above rules are defined in terms of component properties rather than the XML Infoset representation of the component model. The following sections specify in detail how the pointer parts are constructed from the component model.

A.2.1 The Description Component

`wsdl.description()`

A.2.2 The Element Declaration Component

`wsdl.elementDeclaration(element)`

`wsdl.elementDeclaration(element,system)`

1. *element* is the {[name](#)} property of the [Element Declaration](#) component.
2. *system* is the namespace absolute IRI of the extension type system used for the [Element Declaration](#) component (see [3.2 Using Other Schema Languages](#)). This parameter is absent if XML Schema is the type system.

A.2.3 The Type Definition Component

`wsdl.typeDefinition(type)`

`wsdl.typeDefinition(type,system)`

1. *type* is the {[name](#)} property of the [Type Definition](#) component.
2. *system* is the namespace absolute IRI of the extension type system used for the [Type Definition](#) component (see [3.2 Using Other Schema Languages](#)). This parameter is absent if XML Schema is the type system.

A.2.4 The Interface Component

`wsdl.interface(interface)`

1. *interface* is the local name of the {[name](#)} property of the [Interface](#) component.

A.2.5 The Interface Fault Component

`wsdl.interfaceFault(interface/fault)`

1. *interface* is the local name of the {[name](#)} property of the parent [Interface](#) component.
2. *fault* is the local name of the {[name](#)} property of the [Interface Fault](#) component.

A.2.6 The Interface Operation Component

`wsdl.interfaceOperation(interface/operation)`

1. *interface* is the local name of the {[name](#)} property of the parent [Interface](#) component.
2. *operation* is the local name of the {[name](#)} property of the [Interface Operation](#) component.

A.2.7 The Interface Message Reference Component

`wsdl.interfaceMessageReference(interface/operation/message)`

1. *interface* is the local name of the {[name](#)} property of the grandparent [Interface](#) component.
2. *operation* is the local name of the {[name](#)} property of the parent [Interface Operation](#) component.
3. *message* is the {[message label](#)} property of the [Interface Message Reference](#) component.

A.2.8 The Interface Fault Reference Component

`wsdl.interfaceFaultReference(interface/operation/message/fault)`

1. *interface* is the local name of the {[name](#)} property of the grandparent [Interface](#) component.
2. *operation* is the local name of the {[name](#)} property of the parent [Interface Operation](#) component.
3. *message* is the {[message label](#)} property of the [Interface Fault Reference](#) component.
4. *fault* is the {[name](#)} property of the [Interface Fault](#) component referred to by the {[interface fault](#)} property of the [Interface Fault Reference](#) component.

A.2.9 The Binding Component

`wsdl.binding(binding)`

1. *binding* is the local name of the {[name](#)} property of the [Binding](#) component.

A.2.10 The Binding Fault Component

`wsdl.bindingFault(binding/fault)`

1. *binding* is the local name of the {[name](#)} property of the parent [Binding](#) component.
2. *fault* is the {[name](#)} property of the [Interface Fault](#) component referred to by the {[interface fault](#)} property of the [Binding Fault](#) component.

A.2.11 The Binding Operation Component

`wsdl.bindingOperation(binding/operation)`

1. *binding* is the local name of the {[name](#)} property of the parent [Binding](#) component.
2. *operation* is the {[name](#)} property of the [Interface Operation](#) component referred to by the {[interface operation](#)} property of the [Binding Operation](#) component.

A.2.12 The Binding Message Reference Component

`wsdl.bindingMessageReference(binding/operation/message)`

1. *binding* is the local name of the {[name](#)} property of the grandparent [Binding](#) component.
2. *operation* is the {[name](#)} property of the [Interface Operation](#) component referred to by the {[interface operation](#)} property of the parent [Binding Operation](#) component.
3. *message* is the {[message label](#)} property of the [Interface Message Reference](#) component referred to by the {[interface message reference](#)} property of the [Binding Message Reference](#) component.

A.2.13 The Binding Fault Reference Component

`wsdl.bindingFaultReference(binding/operation/message/fault)`

1. *binding* is the local name of the {[name](#)} property of the grandparent [Binding](#) component.
2. *operation* is the {[name](#)} property of the [Interface Operation](#) component referred to by the {[interface operation](#)} property of the parent [Binding Operation](#) component.
3. *message* is the {[message label](#)} property of the [Interface Fault Reference](#) component referred to by the {[interface fault reference](#)} property of the [Binding Fault Reference](#) component.
4. *fault* is the {[name](#)} property of the [Interface Fault](#) component referred to by the {[interface fault](#)} property of the [Interface Fault Reference](#) component referred to by the {[interface fault reference](#)} property of the [Binding Fault Reference](#) component.

A.2.14 The Service Component

`wsdl.service(service)`

1. *service* is the local name of the {[name](#)} property of the [Service](#) component.

A.2.15 The Endpoint Component

`wsdl.endpoint(service/endpoint)`

1. *service* is the local name of the `{name}` property of the parent [Service](#) component.
2. *endpoint* is the `{name}` property of the [Endpoint](#) component.

A.2.16 Extension Components

WSDL 2.0 is extensible and it is possible for an extension to define new components types. The XPointer Framework scheme for extension components is:

`wSDL:extension(namespace, identifier)`

1. *namespace* is the namespace URI that identifies the extension, e.g. for the WSDL 2.0 SOAP 1.2 Binding the namespace is <http://www.w3.org/ns/wSDL/soap>.
2. *identifier* is defined by the extension using a syntax specific to the extension. The owner of the extension must define any components contributed by the extension and a syntax for identifying them.

A.3 Security considerations

This media type uses the "+xml" convention, it shares the same security considerations as described in [\[IETF RFC 3023\]](#), section 10.

B. Acknowledgements (Non-Normative)

This document is the work of the [W3C Web Service Description Working Group](#).

Members of the Working Group are (at the time of writing, and by alphabetical order): Charlton Barreto (Adobe Systems, Inc), Allen Brookes (Rogue Wave Software), Dave Chappell (Sonic Software), Helen Chen (Agfa-Gevaert N. V.), Roberto Chinnici (Sun Microsystems), Kendall Clark (University of Maryland), Glen Daniels (Sonic Software), Paul Downey (British Telecommunications), Youenn Fablet (Canon), Ram Jeyaraman (Microsoft), Tom Jordahl (Adobe Systems), Anish Karmarkar (Oracle Corporation), Jacek Kopecky (DERI Innsbruck at the Leopold-Franzens-Universität Innsbruck, Austria), Amelia Lewis (TIBCO Software, Inc.), Philippe Le Hegaret (W3C), Michael Liddy (Education.au Ltd.), Kevin Canyang Liu (SAP AG), Jonathan Marsh (WSO2), Monica Martin (Sun Microsystems), Josephine Micallef (SAIC - Telcordia Technologies), Jeff Mischinsky (Oracle Corporation), Dale Moberg (Cyclone Commerce), Jean-Jacques Moreau (Canon), David Orchard (BEA Systems, Inc.), Gilbert Pilz (BEA Systems, Inc.), Tony Rogers (Computer Associates), Arthur Ryman (IBM), Adi Sakala (IONA Technologies), Michael Shepherd (Xerox), Asir Vedamuthu (Microsoft Corporation), Sanjiva Weerawarana (WSO2), Ümit Yalçınalp (SAP AG), Peter Zehler (Xerox).

Previous members were: Eran Chinthaka (WSO2), Mark Nottingham (BEA Systems, Inc.), Hugo Haas (W3C), Vivek Pandey (Sun Microsystems), Bijan Parsia (University of Maryland), Lily Liu (webMethods, Inc.), Don Wright (Lexmark), Joyce Yang (Oracle Corporation), Daniel Schutzer (Citigroup), Dave Solo (Citigroup), Stefano Pogliani (Sun Microsystems), William Stumbo (Xerox), Stephen White (SeeBeyond), Barbara Zengler (DaimlerChrysler Research and Technology), Tim Finin (University of Maryland), Laurent De Teneuille (L'Echangeur), Johan Pahlsson (L'Echangeur), Mark Jones (AT&T), Steve Lind (AT&T), Sandra Swearingen (U.S. Department of Defense, U.S. Air Force), Philippe Le Hégarret (W3C), Jim Hendler (University of Maryland), Dietmar Gaertner (Software AG), Michael Champion (Software AG), Don Mullen (TIBCO Software, Inc.), Steve Graham (Global Grid Forum), Steve Tuecke (Global Grid Forum), Michael Mahan (Nokia), Bryan Thompson (Hicks & Associates), Ingo Melzer (DaimlerChrysler Research and Technology), Sandeep Kumar (Cisco Systems), Alan Davies (SeeBeyond), Jacek Kopecky (Systinet), Mike Ballantyne (Electronic Data Systems), Mike Davoren (W. W. Grainger), Dan Kulp (IONA Technologies), Mike McHugh (W. W. Grainger), Michael Mealling (Verisign), Waqar Sadiq (Electronic Data Systems), Yaron Goland (BEA Systems, Inc.), Ümit Yalçınalp (Oracle Corporation), Peter Madziak (Agfa-Gevaert N. V.), Jeffrey Schlimmer (Microsoft Corporation), Hao He (The Thomson Corporation), Erik Ackerman (Lexmark), Jerry Thrasher (Lexmark), Prasad Yendluri (webMethods, Inc.), William Vambenepe (Hewlett-Packard Company), David Booth (W3C), Sanjiva Weerawarana (IBM), Asir Vedamuthu (webMethods, Inc.), Igor Sedukhin (Computer Associates), Martin Gudgin (Microsoft Corporation), Rebecca Bergersen (IONA Technologies), Ugo Corda (SeeBeyond).

The people who have contributed to [discussions on www-ws-desc@w3.org](#) are also gratefully acknowledged.

C. IRI-References for WSDL 2.0 Components (Non-Normative)

This appendix provides a syntax for IRI-references for all components found in a WSDL 2.0 document. The IRI-references are easy to understand and compare, while imposing no burden on the WSDL 2.0 author.

C.1 WSDL 2.0 IRIs

There are two main cases for WSDL 2.0 IRIs:

- the IRI of a WSDL 2.0 document
- the IRI of a WSDL 2.0 namespace

The IRI of a WSDL 2.0 document can be dereferenced to give a resource representation that contributes component definitions to a single WSDL 2.0 namespace. If the media type is set to the WSDL 2.0 media type, then the fragment identifiers can be used to identify the main components that are defined in the document.

However, in keeping with the recommendation in [2.1.1 The Description Component](#) that the namespace URI be dereferencable to a WSDL 2.0 document, this appendix specifies the use of the namespace IRI with the WSDL 2.0 fragment identifiers to form an IRI-reference.

The IRI in an IRI-reference for a WSDL 2.0 component is the namespace name of the `{name}` property of either the component itself, in the case of [Interface](#), [Binding](#), and [Service](#) components, or the `{name}` property of the ancestor top-level component. The IRI provided by the namespace name of the `{name}` property is combined with a zero or more `xmlns` pointer parts (see [3.4 Namespace Binding Context](#) in [\[XPointer\]](#)) followed by a single WSDL 2.0 pointer part as defined in [A.2 Fragment Identifiers](#).

C.2 Canonical Form for WSDL 2.0 Component Designators

The IRI-references described above MAY be used as WSDL 2.0 component designators. For ease of comparison, the fragment identifier of WSDL 2.0 component designators SHOULD conform to the following canonicalization rules:

- The fragment identifier consists of a sequence zero or more `xmlns()` pointer parts followed by exactly one `wsdl.*()` pointer part. [†]
- Each `xmlns()` pointer part that appears in the fragment identifier defines a namespace that is referenced by the `wsdl.*()` pointer part. [†]
- Each `xmlns()` pointer part defines a unique namespace. [†]
- The `xmlns()` pointer parts define namespaces in the same order as they are referenced in the `wsdl.*()` pointer part. [†]
- The namespace prefixes defined by the `xmlns()` pointer parts are named `ns1`, `ns2`, etc., in the order of their appearance. [†]
- The fragment identifier contains no optional whitespace. [†]
- No `xmlns()` pointer part defines a namespace for the `targetNamespace` of the WSDL 2.0 document. [†]

C.3 Example

Consider the following WSDL 2.0 document located at <http://example.org/TicketAgent.wsdl>:

Example C-1. IRI-References - Example WSDL 2.0 Document

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:description
  targetNamespace="http://example.org/TicketAgent.wsdl20"
  xmlns:xsTicketAgent="http://example.org/TicketAgent.xsd"
  xmlns:wsdl="http://www.w3.org/ns/wsdl"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/ns/wsdl http://www.w3.org/2007/06/wsdl/wsdl20.xsd">

  <wsdl:types>
    <xs:import schemaLocation="TicketAgent.xsd"
      namespace="http://example.org/TicketAgent.xsd" />
  </wsdl:types>

  <wsdl:interface name="TicketAgent">
    <wsdl:operation name="listFlights"
      pattern="http://www.w3.org/ns/wsdl/in-out">
      <wsdl:input element="xsTicketAgent:listFlightsRequest"/>
      <wsdl:output element="xsTicketAgent:listFlightsResponse"/>
    </wsdl:operation>

    <wsdl:operation name="reserveFlight"
      pattern="http://www.w3.org/ns/wsdl/in-out">
      <wsdl:input element="xsTicketAgent:reserveFlightRequest"/>
      <wsdl:output element="xsTicketAgent:reserveFlightResponse"/>
    </wsdl:operation>
  </wsdl:interface>
</wsdl:description>
```

Its components have the following IRI-references which follow the above canonicalization rules except for the presence of optional whitespace that has been added in order to improve the formatting:

Example C-2. IRI-References - Example IRIs

```
http://example.org/TicketAgent.wsdl20#
wsdl.description()

http://example.org/TicketAgent.wsdl20#
xmlns(ns1=http://example.org/TicketAgent.xsd)
wsdl.elementDeclaration(ns1:listFlightsRequest)

http://example.org/TicketAgent.wsdl20#
```

```

xmlns(ns1=http://example.org/TicketAgent.xsd)
wsdl:elementDeclaration(ns1:listFlightsResponse)

http://example.org/TicketAgent.wsdl20#
xmlns(ns1=http://example.org/TicketAgent.xsd)
wsdl:elementDeclaration(ns1:reserveFlightRequest)

http://example.org/TicketAgent.wsdl20#
xmlns(ns1=http://example.org/TicketAgent.xsd)
wsdl:elementDeclaration(ns1:reserveFlightResponse)

http://example.org/TicketAgent.wsdl20#
wsdl:interface(TicketAgent)

http://example.org/TicketAgent.wsdl20#
wsdl:interfaceOperation(TicketAgent/listFlights)

http://example.org/TicketAgent.wsdl20#
wsdl:interfaceMessageReference(TicketAgent/listFlights/In)

http://example.org/TicketAgent.wsdl20#
wsdl:interfaceMessageReference(TicketAgent/listFlights/Out)

http://example.org/TicketAgent.wsdl20#
wsdl:interfaceOperation(TicketAgent/reserveFlight)

http://example.org/TicketAgent.wsdl20#
wsdl:interfaceMessageReference(TicketAgent/reserveFlight/In)

http://example.org/TicketAgent.wsdl20#
wsdl:interfaceMessageReference(TicketAgent/reserveFlight/Out)

```

D. Component Summary (Non-Normative)

[Table D-1](#) lists all the components in the WSDL 2.0 abstract Component Model, and all their properties. Note some properties have a generic definition that is used in more than one component. In this case, the Component column contains a "-" to indicate this generic definition of the property.

Table D-1. Summary of WSDL 2.0 Components and their Properties

Component	Defined Properties
-	{ name }, { parent }
Binding	{ binding faults }, { binding operations }, { interface }, { name }, { type }
Binding Fault	{ interface fault }, { parent }
Binding Fault Reference	{ interface fault reference }, { parent }
Binding Message Reference	{ interface message reference }, { parent }
Binding Operation	{ binding fault references }, { binding message references }, { interface operation }, { parent }
Description	{ bindings }, { element declarations }, { interfaces }, { services }, { type definitions }
Element Declaration	{ name }, { system }
Endpoint	{ address }, { binding }, { name }, { parent }
Interface	{ extended interfaces }, { interface faults }, { interface operations }, { name }
Interface Fault	{ element declaration }, { message content model }, { name }, { parent }
Interface Fault Reference	{ direction }, { interface fault }, { message label }, { parent }
Interface Message Reference	{ direction }, { element declaration }, { message content model }, { message label }, { parent }
Interface Operation	{ interface fault references }, { interface message references }, { message exchange pattern }, { name }, { parent }, { style }
Service	{ endpoints }, { interface }, { name }
Type Definition	{ name }, { system }
Property	Where Defined
address	Endpoint.{ address }
binding	Endpoint.{ binding }
binding fault references	Binding Operation.{ binding fault references }
binding faults	Binding.{ binding faults }
binding message references	Binding Operation.{ binding message references }
binding operations	Binding.{ binding operations }

bindings	Description.{ bindings }
direction	Interface Fault Reference.{ direction }, Interface Message Reference.{ direction }
element declaration	Interface Fault.{ element declaration }, Interface Message Reference.{ element declaration }
element declarations	Description.{ element declarations }
endpoints	Service.{ endpoints }
extended interfaces	Interface.{ extended interfaces }
interface	Binding.{ interface }, Service.{ interface }
interface fault	Binding Fault.{ interface fault }, Interface Fault Reference.{ interface fault }
interface fault reference	Binding Fault Reference.{ interface fault reference }
interface fault references	Interface Operation.{ interface fault references }
interface faults	Interface.{ interface faults }
interface message reference	Binding Message Reference.{ interface message reference }
interface message references	Interface Operation.{ interface message references }
interface operation	Binding Operation.{ interface operation }
interface operations	Interface.{ interface operations }
interfaces	Description.{ interfaces }
message content model	Interface Fault.{ message content model }, Interface Message Reference.{ message content model }
message exchange pattern	Interface Operation.{ message exchange pattern }
message label	Interface Fault Reference.{ message label }, Interface Message Reference.{ message label }
name	.{ name }, Binding.{ name }, Element Declaration.{ name }, Endpoint.{ name }, Interface.{ name }, Interface Fault.{ name }, Interface Operation.{ name }, Service.{ name }, Type Definition.{ name }
parent	.{ parent }, Binding Fault.{ parent }, Binding Fault Reference.{ parent }, Binding Message Reference.{ parent }, Binding Operation.{ parent }, Endpoint.{ parent }, Interface Fault.{ parent }, Interface Fault Reference.{ parent }, Interface Message Reference.{ parent }, Interface Operation.{ parent }
services	Description.{ services }
style	Interface Operation.{ style }
system	Element Declaration.{ system }, Type Definition.{ system }
type	Binding.{ type }
type definitions	Description.{ type definitions }

E. Assertion Summary (Non-Normative)

This appendix summarizes assertions about WSDL 2.0 documents and components that are not enforced by the WSDL 2.0 schema. Each assertion is assigned a unique identifier which WSDL 2.0 processors may use to report errors.

Table E-1. Summary of Assertions about WSDL 2.0 Documents

Id	Assertion
Description-1004	If a WSDL 2.0 document is split into multiple WSDL 2.0 documents (which may be combined as needed via 4.1 Including Descriptions), then the <code>targetNamespace</code> <i>attribute information item</i> SHOULD resolve to a master WSDL 2.0 document that includes all the WSDL 2.0 documents needed for that service description.
Description-1005	Zero or more <i>element information items</i> amongst its [children], in order as follows:
Description-1006	Its value MUST be an absolute IRI (see [IETF RFC 3987]) and should be dereferencable.
Import-1082	As with XML schema, any WSDL 2.0 document that references a foreign component MUST have a <code>wsdl:import</code> <i>element information item</i> for the associated foreign namespace (but which does not necessarily provide a <code>location</code> <i>attribute information item</i> that identifies the WSDL 2.0 document in which the referenced component is defined).
Import-1083	If a WSDL 2.0 document contains more than one <code>wsdl:import</code> <i>element information item</i> for a given value of the <code>namespace</code> <i>attribute information item</i> , then they MUST provide different values for the <code>location</code> <i>attribute information item</i> .
Import-1084	This value MUST NOT match the actual value of <code>targetNamespace</code> <i>attribute information</i>

	<i>item</i> in the enclosing WSDL 2.0 document.
Import-1085	If the location attribute in the <code>import</code> <i>element information item</i> is dereferencable, then it MUST reference a WSDL 2.0 document.
Import-1086	If the location <i>attribute information item</i> of the <code>import</code> <i>element information item</i> is dereferencable, then the actual value of the namespace <i>attribute information item</i> MUST be identical to the actual value of the targetNamespace <i>attribute information item</i> of the referenced WSDL 2.0 document (see 7. Locating WSDL 2.0 Documents).
Include-1080	The IRI indicated by <code>location</code> MUST resolve to a WSDL 2.0 document.
Include-1081	The actual value of the targetNamespace <i>attribute information item</i> of the included WSDL 2.0 document MUST match the actual value of the targetNamespace <i>attribute information item</i> of the description <i>element information item</i> which is the [parent] of the <code>include</code> <i>element information item</i> .
Interface-1012	Its value, if present, MUST contain absolute IRIs (see [IETF RFC 3987]).
InterfaceFault-1017	If the <code>element</code> <i>attribute information item</i> has a value, then it MUST resolve to an Element Declaration component from the { element declarations } property of the Description component.
InterfaceFaultReference-1040	The <code>messageLabel</code> <i>attribute information item</i> MUST be present in the XML representation of an Interface Fault Reference component with a given { direction }, if the { message exchange pattern } of the parent Interface Operation component has more than one fault with that direction.
InterfaceMessageReference-1036	If the <code>element</code> <i>attribute information item</i> has a value, then it MUST resolve to an Element Declaration component from the { element declarations } property of the Description component.
Location-1092	It MUST NOT appear on a <code>wsdl:description</code> element or any of its children/descendants.
Location-1094	For each pair of IRIs, if the location IRI of the pair is dereferencable, then it MUST reference a WSDL 2.0 (or 1.1) document whose target namespace is the namespace IRI of the pair.
MessageLabel-1030	If the <code>messageLabel</code> <i>attribute information item</i> of an interface message reference <i>element information item</i> is present, then its actual value MUST match the { message label } of some placeholder message with { direction } equal to the message direction.
MessageLabel-1031	If the <code>messageLabel</code> <i>attribute information item</i> of an interface message reference <i>element information item</i> is absent then there MUST be a unique placeholder message with { direction } equal to the message direction.
MessageLabel-1032	If the local name is <code>input</code> then the message exchange pattern MUST have at least one placeholder message with direction "In".
MessageLabel-1033	If the local name is <code>output</code> then the message exchange pattern MUST have at least one placeholder message with direction "Out".
MessageLabel-1034	If the local name is <code>infault</code> then the message exchange pattern MUST support at least one fault in the "In" direction.
MessageLabel-1035	If the local name is <code>outfault</code> then the message exchange pattern MUST support at least one fault in the "Out" direction.
MessageLabel-1041	The <code>messageLabel</code> <i>attribute information item</i> of an interface fault reference <i>element information item</i> MUST be present if the message exchange pattern has more than one placeholder message with { direction } equal to the message direction.
MessageLabel-1042	If the <code>messageLabel</code> <i>attribute information item</i> of an interface fault reference <i>element information item</i> is present then its actual value MUST match the { message label } of some placeholder message with { direction } equal to the message direction.
MessageLabel-1043	If the <code>messageLabel</code> <i>attribute information item</i> of an interface fault reference <i>element information item</i> is absent then there MUST be a unique placeholder message with { direction } equal to the message direction.
MessageLabel-1053	If the <code>messageLabel</code> <i>attribute information item</i> of a binding message reference <i>element information item</i> is present then its actual value MUST match the { message label } of some placeholder message with { direction } equal to the message direction.
MessageLabel-1054	If the <code>messageLabel</code> <i>attribute information item</i> of a binding message reference <i>element information item</i> is absent then there MUST be a unique placeholder message with { direction } equal to the message direction.
MessageLabel-1056	The <code>messageLabel</code> <i>attribute information item</i> of a binding fault reference <i>element information item</i> MUST be present if the message exchange pattern has more than one placeholder message with { direction } equal to the message direction.
MessageLabel-1057	If the <code>messageLabel</code> <i>attribute information item</i> of a binding fault reference <i>element information item</i> is present then its actual value MUST match the { message label } of some placeholder message with { direction } equal to the message direction.
MessageLabel-1058	If the <code>messageLabel</code> <i>attribute information item</i> of a binding fault reference <i>element information item</i> is absent then there MUST be a unique placeholder message with

	{ direction } equal to the message direction.
QName-resolution-1064	A Description component MUST NOT have such broken references.
Schema-1066	A WSDL 2.0 document MUST NOT refer to XML Schema components in a given namespace UNLESS an <code>xs:import</code> or <code>xs:schema</code> <i>element information item</i> for that namespace is present OR the namespace is the XML Schema namespace, http://www.w3.org/2001/XMLSchema , which contains built-in types as defined in XML Schema Part 2: Datatypes Second Edition [XML Schema: Datatypes].
Schema-1069	The referenced schema MUST contain a <code>targetNamespace</code> <i>attribute information item</i> on its <code>xs:schema</code> <i>element information item</i> .
Schema-1070	The value of the <code>targetNamespace</code> <i>attribute information item</i> of the <code>xs:schema</code> <i>element information item</i> of an imported schema MUST equal the value of the namespace of the <code>import</code> <i>element information item</i> in the importing WSDL 2.0 document.
Schema-1073	A WSDL 2.0 document MUST NOT define the same element or type in more than one inlined schema.
Schema-1075	A specification of extension syntax for an alternative schema language MUST use a namespace that is different than the namespace of XML Schema.
Schema-1076	The namespace used for an alternate schema language MUST be an absolute IRI.
Schema-1079	If <code>wsdlx:interface</code> and <code>wsdlx:binding</code> are used together then they MUST satisfy the same consistency rules that apply to the { interface } property of a Service component and the { binding } property of a nested Endpoint component, that is either the binding refers the interface of the service or the binding refers to no interface.
Types-1074	A specification of extension syntax for an alternative schema language MUST include the declaration of an <i>element information item</i> , intended to appear as a child of the <code>wsdl:types</code> <i>element information item</i> , which references, names, and locates the schema instance (an <code>import</code> <i>element information item</i>).
Types-1077	The type of the <code>wsdlx:interface</code> <i>attribute information item</i> is an <code>xs:QName</code> that specifies the { name } property of an Interface component.
Types-1078	The type of the <code>wsdlx:binding</code> <i>attribute information item</i> is an <code>xs:QName</code> that specifies the { name } property of a Binding component.

Table E-2. Summary of Assertions about WSDL 2.0 Components

Id	Assertion
Binding-1044	If a Binding component specifies any operation-specific binding details (by including Binding Operation components) or any fault binding details (by including Binding Fault components), then it MUST specify an interface the Binding component applies to, so as to indicate which interface the operations come from.
Binding-1045	A Binding component that defines bindings for an Interface component MUST define bindings for all the operations of that Interface component.
Binding-1046	Similarly, whenever a reusable Binding component (i.e. one that does not specify an Interface component) is applied to a specific Interface component in the context of an Endpoint component (see 2.13.1 The Endpoint Component), the Binding component MUST define bindings for each Interface Operation and Interface Fault component of the Interface component, via a combination of properties defined on the Binding component itself and default binding rules specific to its binding type.
Binding-1047	A Binding component that defines bindings for an Interface component MUST define bindings for all the faults of that Interface component that are referenced from any of the operations in that Interface component.
Binding-1048	This <code>xs:anyURI</code> MUST be an absolute IRI as defined by [IETF RFC 3987].
Binding-1049	For each Binding component in the { bindings } property of a Description component, the { name } property MUST be unique.
BindingFault-1050	For each Binding Fault component in the { binding faults } property of a Binding component, the { interface fault } property MUST be unique.
BindingFaultReference-1055	For each Binding Fault Reference component in the { binding fault references } property of a Binding Operation component, the { interface fault reference } property MUST be unique.
BindingFaultReference-1059	There MUST be an Interface Fault Reference component in the { interface fault references } of the Interface Operation being bound with { message label } equal to the effective message label and with { interface fault } equal to an Interface Fault component with { name } equal to the actual value of the <code>ref</code> <i>attribute information item</i> .
BindingMessageReference-1052	For each Binding Message Reference component in the { binding message references } property of a Binding Operation component, the { interface message reference } property MUST be unique.
BindingOperation-1051	For each Binding Operation component in the { binding operations } property of a Binding component, the { interface operation } property MUST be unique.
CanonFragId-1097	The fragment identifier consists of a sequence zero or more <code>xmlns()</code> pointer parts

	followed by exactly one <code>wsdl.*()</code> pointer part.
CanonFragId-1098	Each <code>xmlns()</code> pointer part that appears in the fragment identifier defines a namespace that is referenced by the <code>wsdl.*()</code> pointer part.
CanonFragId-1099	Each <code>xmlns()</code> pointer part defines a unique namespace.
CanonFragId-1100	The <code>xmlns()</code> pointer parts define namespaces in the same order as they are referenced in the <code>wsdl.*()</code> pointer part.
CanonFragId-1101	The namespace prefixes defined by the <code>xmlns()</code> pointer parts are named <code>ns1</code> , <code>ns2</code> , etc., in the order of their appearance.
CanonFragId-1102	The fragment identifier contains no optional whitespace.
CanonFragId-1103	No <code>xmlns()</code> pointer part defines a namespace for the <code>targetNamespace</code> of the WSDL 2.0 document.
Compare-URI-IRI-1065	When such absolute URIs and IRIs are being compared to determine equivalence (see 2.15 Equivalence of Components), they MUST be compared character-by-character as indicated in [IETF RFC 3987] .
Description-1001	The value of the <code>targetNamespace</code> <i>attribute information item</i> SHOULD be dereferencable.
Description-1002	It SHOULD resolve to a human or machine processable document that directly or indirectly defines the intended semantics of those components.
Description-1003	It MAY resolve to a WSDL 2.0 document that provides service description information for that namespace.
Description-1067	For each component in the imported namespace, a corresponding Element Declaration component or Type Definition component MUST appear in the <code>{element declarations}</code> or <code>{type definitions}</code> property respectively of the Description component corresponding to the WSDL document that imports the schema, or that imports directly or indirectly a WSDL document that imports the schema.
Description-1068	Schema components not in an imported namespace MUST NOT appear in the <code>{element declarations}</code> or <code>{type definitions}</code> properties.
Description-1071	For each component defined and declared in the inlined schema document or included by <code>xs:include</code> , a corresponding Element Declaration component or Type Definition component MUST appear in the <code>{element declarations}</code> property or <code>{type definitions}</code> property respectively of the Description component corresponding to the WSDL document that contains the schema, or that imports directly or indirectly a WSDL document that contains the schema.
Description-1072	Schema components not defined or declared in the inlined schema document or included by <code>xs:include</code> MUST NOT appear in the <code>{element declarations}</code> or <code>{type definitions}</code> properties.
Endpoint-1061	This <code>xs:anyURI</code> MUST be an absolute IRI as defined by [IETF RFC 3987] .
Endpoint-1062	For each Endpoint component in the <code>{endpoints}</code> property of a Service component, the <code>{binding}</code> property MUST either be a Binding component with an unspecified <code>{interface}</code> property or a Binding component with an <code>{interface}</code> property equal to the <code>{interface}</code> property of the Service component.
Equivalence-1063	Extension properties which are not string values, sets of strings or references MUST describe their values' equivalence rules.
Extensibility-1089	An extension that is NOT marked as mandatory MUST NOT invalidate the meaning of any part of a WSDL 2.0 document.
Extensibility-1090	If a WSDL 2.0 document declares an extension as optional (i.e., NON-mandatory), then the Web service MUST NOT assume that the client supports that extension <i>unless</i> the Web service knows (through some other means) that the client has in fact elected to engage and support that extension.
Extensibility-1091	Therefore, the Web service MUST support every extension that is declared as optional in the WSDL 2.0 document, in addition to supporting every extension that is declared as mandatory.
Extension-1088	The meaning of an extension SHOULD be defined (directly or indirectly) in a document that is available at its namespace IRI.
FragId-1095	For QNames, any prefix MUST be defined by a preceding <code>xmlns</code> pointer part.
FragId-1096	The fragment identifier in a WSDL 2.0 component IRI-reference MUST resolve to some component as defined by the construction rules in Table A-1 .
ImportInclude-1087	The semantics of an extension MUST NOT depend on how components are brought into a component model instance via <code><import></code> or <code><include></code> .
Interface-1009	To avoid circular definitions, an interface MUST NOT appear in the set of interfaces it extends, either directly or indirectly.
Interface-1010	For each Interface component in the <code>{interfaces}</code> property of a Description component, the <code>{name}</code> property MUST be unique.
Interface-1011	The list of <code>xs:QName</code> in an <code>extends</code> <i>attribute information item</i> MUST NOT contain

	duplicates.
InterfaceFault-1013	An <i>xs:token</i> with one of the values <i>#any</i> , <i>#none</i> , <i>#other</i> , or <i>#element</i> .
InterfaceFault-1014	When the <i>{message content model}</i> property has the value <i>#any</i> or <i>#none</i> the <i>{element declaration}</i> property MUST be empty.
InterfaceFault-1015	In cases where, due to an interface extending one or more other interfaces, two or more Interface Fault components have the same value for their <i>{name}</i> property, then the component models of those Interface Fault components MUST be equivalent (see 2.15 Equivalence of Components).
InterfaceFault-1016	For the above reason, it is considered good practice to ensure, where necessary, that the local name of the <i>{name}</i> property of Interface Fault components within a namespace SHOULD be unique, thus allowing such derivation to occur without inadvertent error.
InterfaceFaultReference-1037	The value of this property MUST match the name of a placeholder message defined by the message exchange pattern.
InterfaceFaultReference-1038	The direction MUST be consistent with the direction implied by the fault propagation ruleset used in the message exchange pattern of the operation.
InterfaceFaultReference-1039	For each Interface Fault Reference component in the <i>{interface fault references}</i> property of an Interface Operation component, the combination of its <i>{interface fault}</i> and <i>{message label}</i> properties MUST be unique.
InterfaceMessageReference-1025	An <i>xs:token</i> with one of the values <i>in</i> or <i>out</i> , indicating whether the message is coming to the service or going from the service, respectively.
InterfaceMessageReference-1026	The direction MUST be the same as the direction of the message identified by the <i>{message label}</i> property in the <i>{message exchange pattern}</i> of the Interface Operation component this is contained within.
InterfaceMessageReference-1027	An <i>xs:token</i> with one of the values <i>#any</i> , <i>#none</i> , <i>#other</i> , or <i>#element</i> .
InterfaceMessageReference-1028	When the <i>{message content model}</i> property has the value <i>#any</i> or <i>#none</i> , the <i>{element declaration}</i> property MUST be empty.
InterfaceMessageReference-1029	For each Interface Message Reference component in the <i>{interface message references}</i> property of an Interface Operation component, its <i>{message label}</i> property MUST be unique.
InterfaceOperation-1018	This <i>xs:anyURI</i> MUST be an absolute IRI (see [IETF RFC 3987]).
InterfaceOperation-1019	These <i>xs:anyURIs</i> MUST be absolute IRIs (see [IETF RFC 3986]).
InterfaceOperation-1020	In cases where, due to an interface extending one or more other interfaces, two or more Interface Operation components have the same value for their <i>{name}</i> property, then the component models of those Interface Operation components MUST be equivalent (see 2.15 Equivalence of Components).
InterfaceOperation-1021	For the above reason, it is considered good practice to ensure, where necessary, that the <i>{name}</i> property of Interface Operation components within a namespace SHOULD be unique, thus allowing such derivation to occur without inadvertent error.
InterfaceOperation-1023	An Interface Operation component MUST satisfy the specification defined by each operation style identified by its <i>{style}</i> property.
Location-1093	Its actual value MUST be a list of pairs of IRIs; where the first IRI of a pair, which MUST be an absolute IRI as defined in [IETF RFC 3987] , indicates a WSDL 2.0 (or 1.1) namespace name, and, the second a hint as to the location of a WSDL 2.0 document defining WSDL 2.0 components (or WSDL 1.1 elements [WSDL 1.1]) for that namespace name.
MEP-1022	A message exchange pattern is itself uniquely identified by an absolute IRI, which is used as the value of the <i>{message exchange pattern}</i> property of the Interface Operation component, and which specifies the fault propagation ruleset that its faults obey.
MessageLabel-1024	The value of this property MUST match the name of a placeholder message defined by the message exchange pattern.
Service-1060	For each Service component in the <i>{services}</i> property of a Description component, the <i>{name}</i> property MUST be unique.
Types-1007	Each XML Schema element declaration MUST have a unique QName.
Types-1008	Each XML Schema type definition MUST have a unique QName.